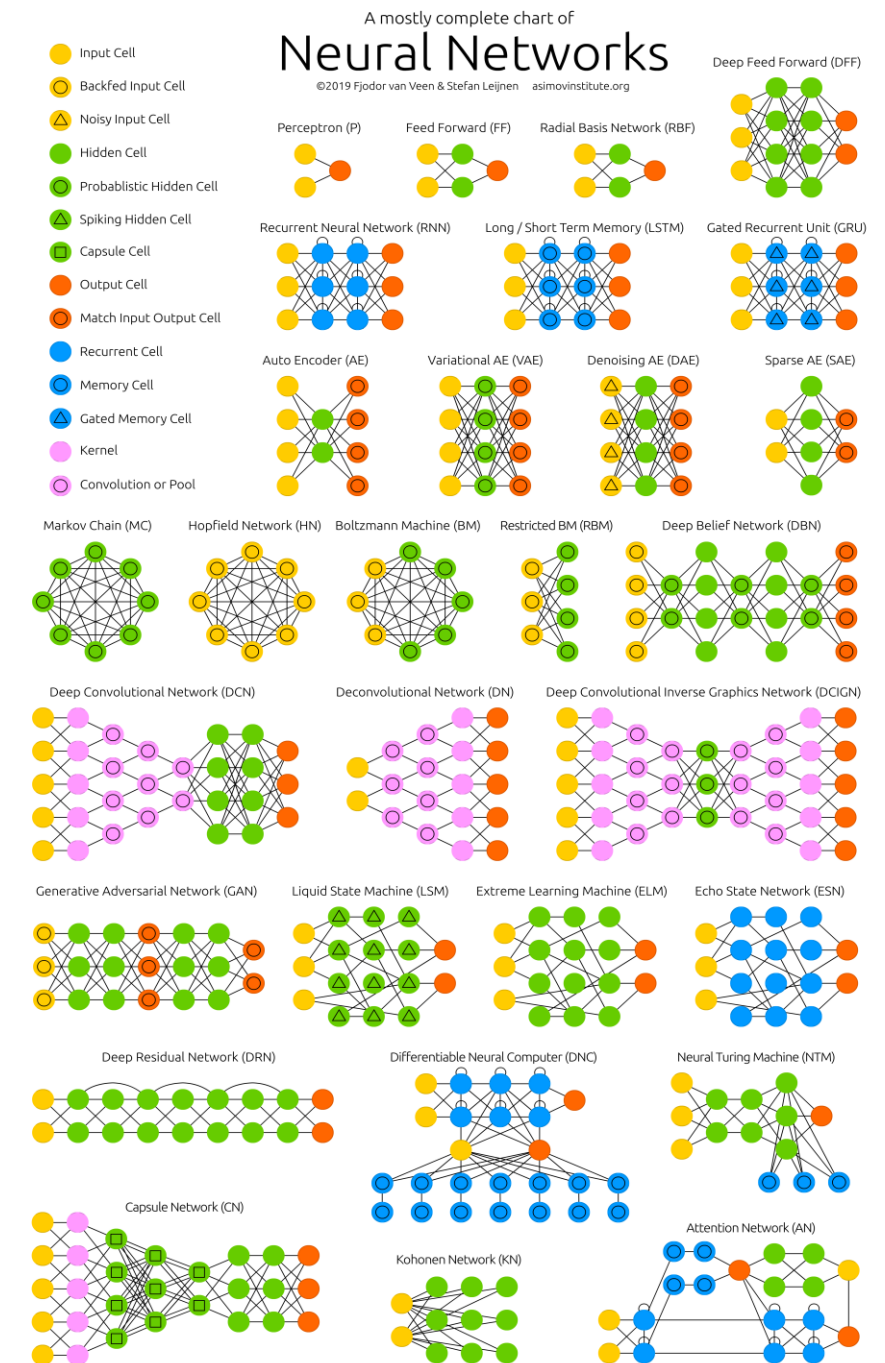


Sesión 11. Autocodificadores – Redes generativas de adversarios

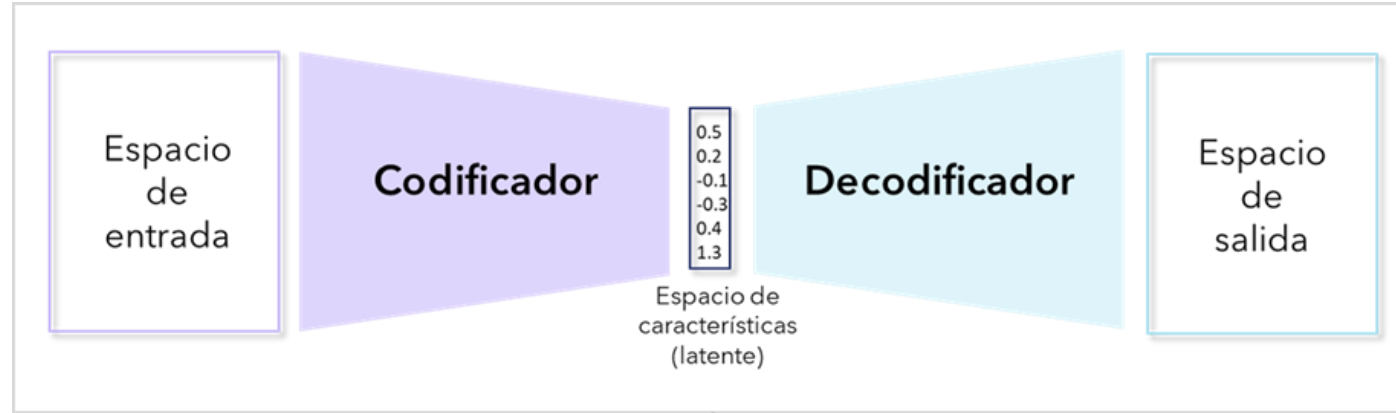
- Los autocodificadores y sus aplicaciones
- Las redes de adversarios.
- Modelos fundacionales.

Características de las redes neuronales

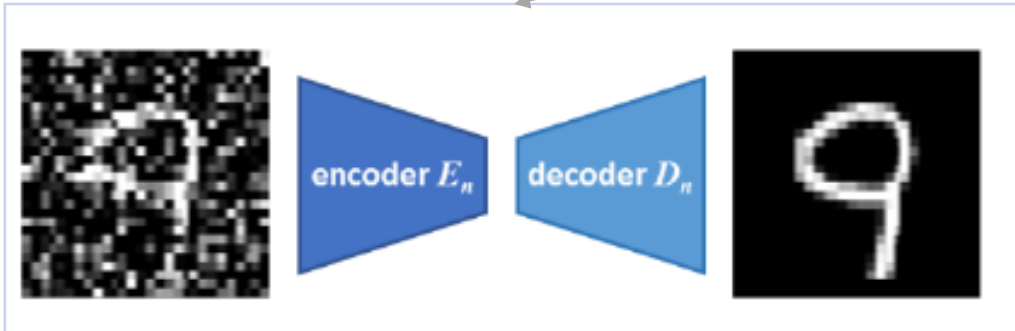
- Arquitectura (define la topología de la red).
 - Estructura por capas (monocapa, multicapa)
 - Patrón de interconexión (determina el flujo de información), por ejemplo: conexiones hacia adelante, conexiones recurrentes, conexiones laterales.
- Tipo de neuronas.
- Tipo de aprendizaje (supervisado, no supervisado, híbrido).
- Regla de aprendizaje (cómo se actualizan los pesos de la red).
- Mecanismos de atención, ...



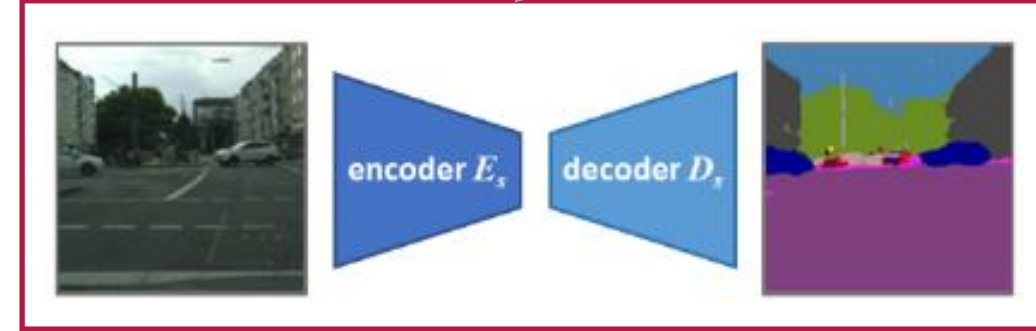
La arquitectura codificador - decodificador



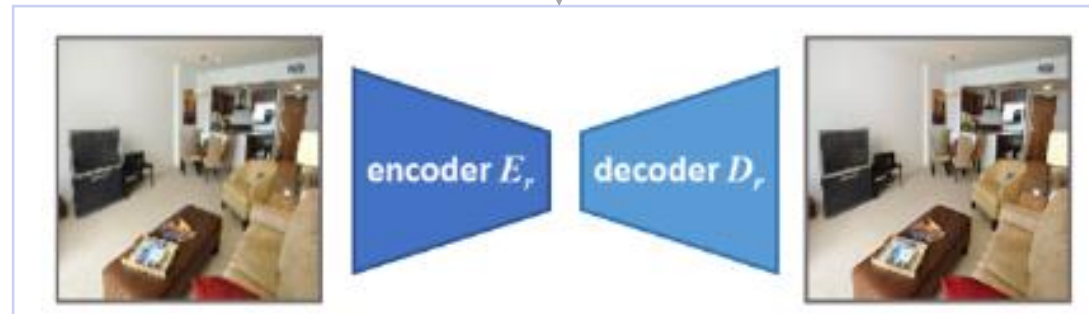
Eliminación de ruido (denoising)



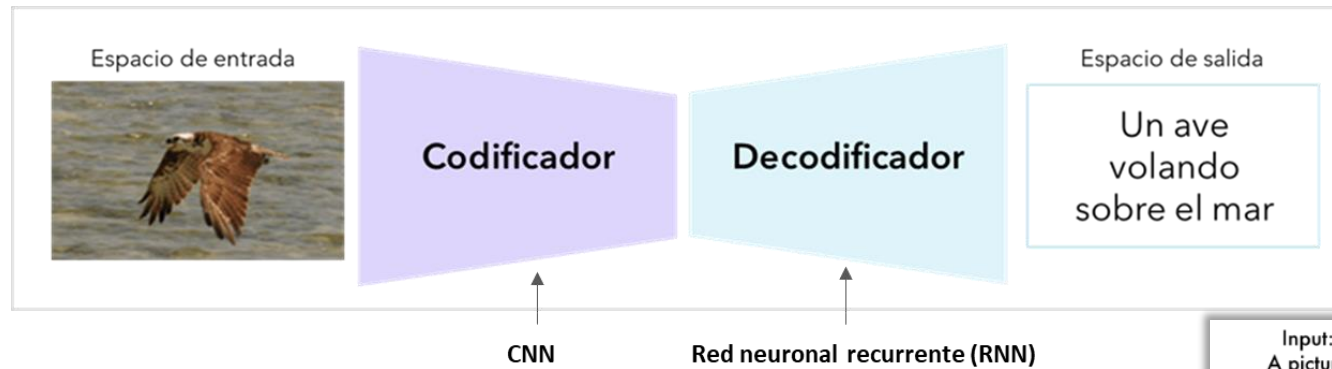
Segmentación semántica



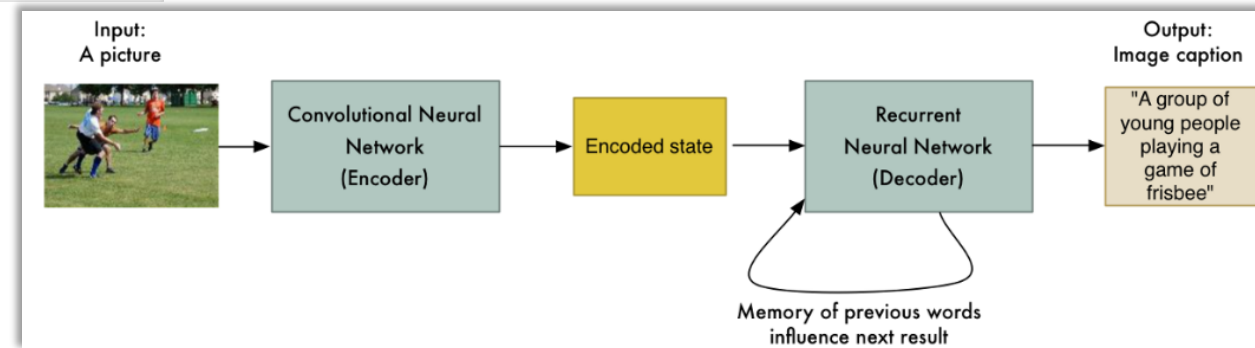
Súper resolución



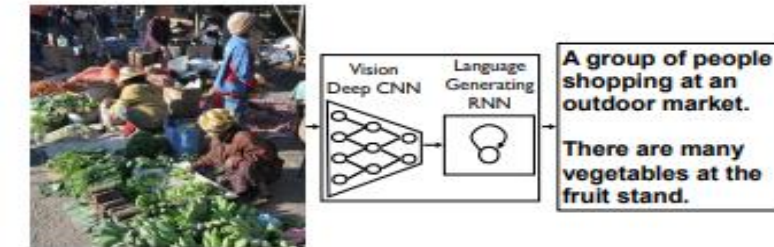
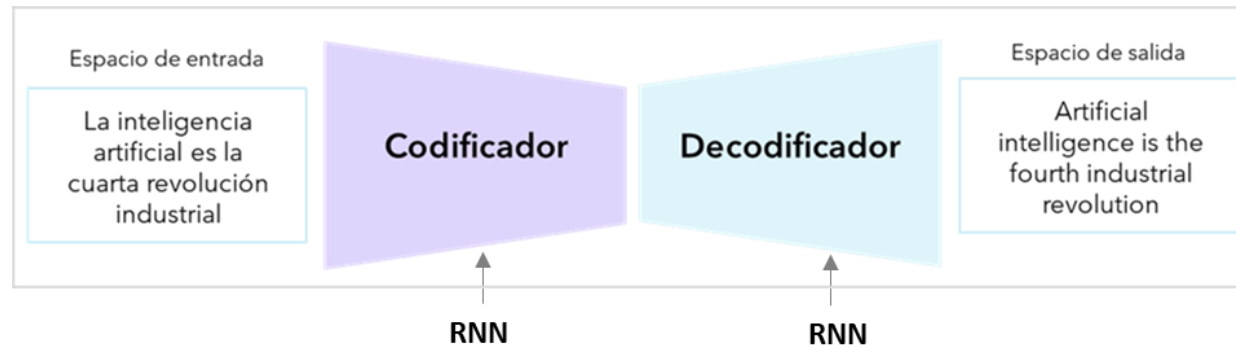
Descripción/titulación de imágenes.



Show and Tell: A Neural Image Caption Generator (Google).
<http://static.googleusercontent.com/media/research.google.com/es//pubs/archive/43274.pdf>



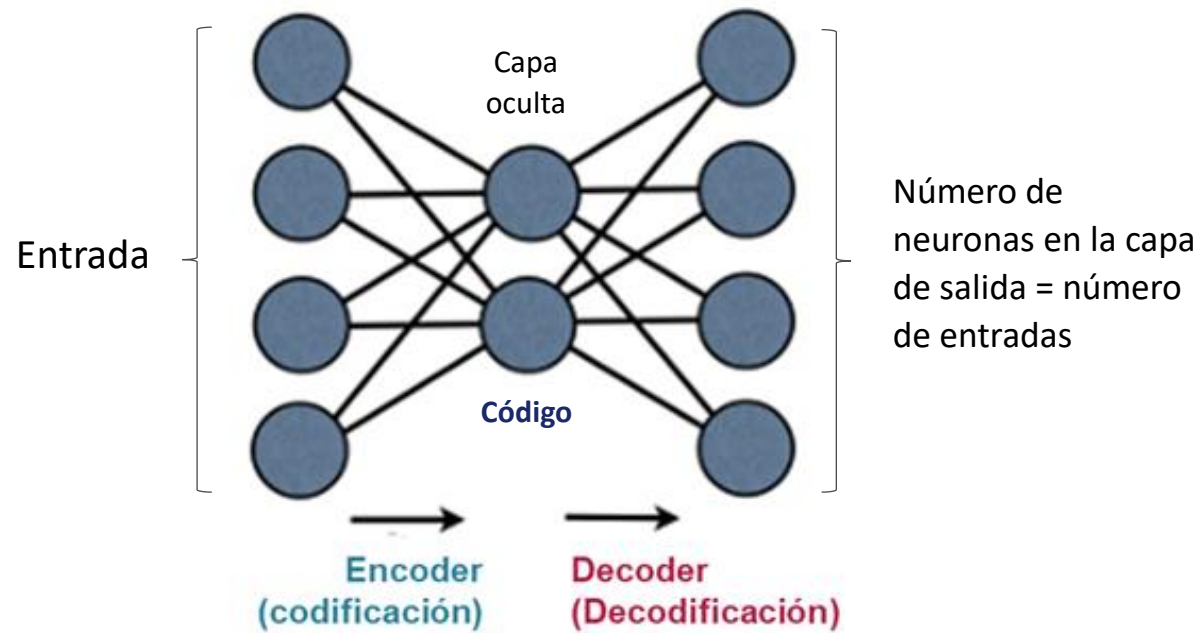
Traslación del lenguaje.



Si el espacio de entrada = espacio de salida $\Rightarrow$ Autocodificador (*Autoencoder*)

❑ Autocodificador estándar

- Un autocodificador (*autoencoder*) es una red neuronal *feedforward* no supervisada, que tiene como objetivo aprender una representación comprimida (codificada) y eficiente de un conjunto de datos.
- Esta representación se conoce con **latente** o código.
- El autocodificador más básico se conoce como *undercomplete (vanilla)* y tiene esta estructura:



- Para lograr el aprendizaje de una nueva representación de los datos, el modelo es entrenado para predecir su propia entrada. Es decir, el conjunto de entrenamiento será:

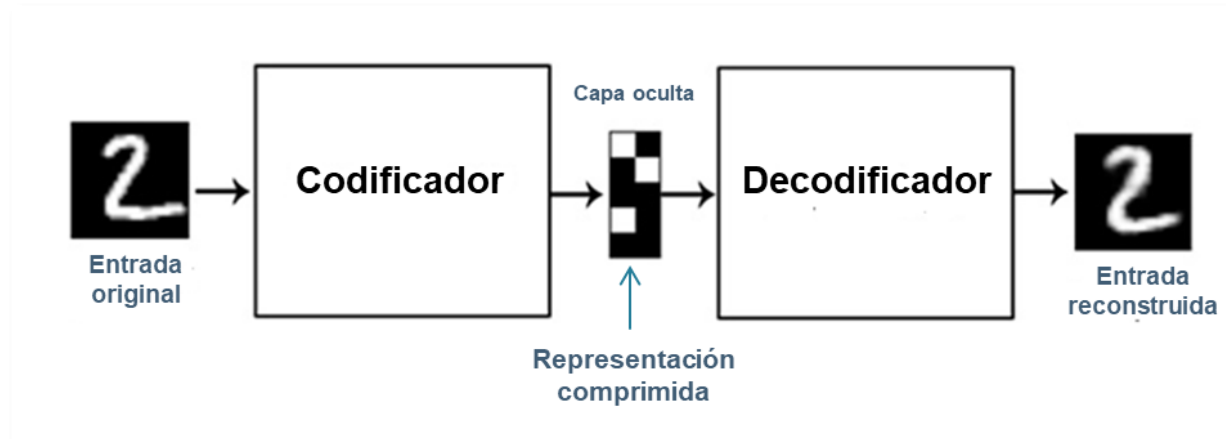
$$D = (x_i)_{i=1..N}$$

Otros tipos de autocodificadores:

- Denoising
- Sparse
- Masked
- Variacional, ...

↑
Modelo generativo

- Al tener como salida target al propio vector $(x_i)_{i=1..N}$, la red se ve obligada a reconstruir la entrada, pero a través de una representación intermedia de baja dimensionalidad.
- Esta nueva representación es creada por la capa oculta. Por ejemplo:



François Chollet. (2021). Deep Learning with Python. Manning Publications.

- Al tener menos neuronas en la capa oculta la red no va a aprender un mapeo de los datos; en vez, captura las características (factores ocultos o latentes) más sobresalientes de estos.
- Idea: Aprender una representación comprimida con mínima pérdida de información:

$$\left. \begin{array}{l} \text{Encoder} \Rightarrow f(x) = c \\ \text{Decoder} \Rightarrow g(c) = \tilde{x} \end{array} \right\} \Rightarrow g(f(x)) = \tilde{x}$$

De esta forma, la capa oculta actúa como un detector de características; es decir, logra aprender las características más importantes de los datos y se logra una reducción de la dimensionalidad. Este nuevo espacio se conoce como **espacio latente (embedding)**.

- Como es un caso especial de las redes *feedforward*, se realiza un entrenamiento con *backpropagation* y función de costo:

$$\text{Función de costo} = J(\mathbf{x}, g(f(\mathbf{x})))$$

Con error cuadrático medio sería:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (x_i - g(f(x_i)))^2$$

Penaliza el modelo cuando las reconstrucciones son diferentes de los datos

- Al establecer como salida target el mismo valor de entrada, el problema del aprendizaje se reduce a tratar de predecir $\mathbf{x}$ a partir de $\mathbf{x}$.

Entrenamiento con BP:

Entrada a la red = $\mathbf{x}$

Salida de la red = $\mathbf{x}_{reconstruida}$

Función de costo

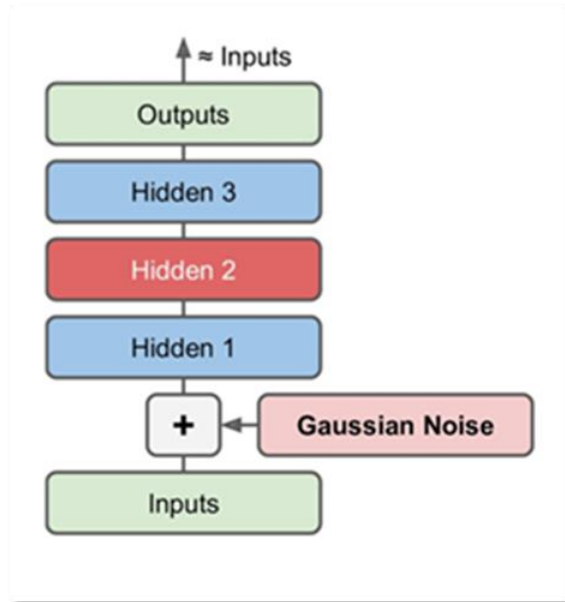
= por ejemplo, MSE (entre $\mathbf{x}$ y $\mathbf{x}_{reconstruida}$)

Funciones de activación = sigmoide, ReLU, ...

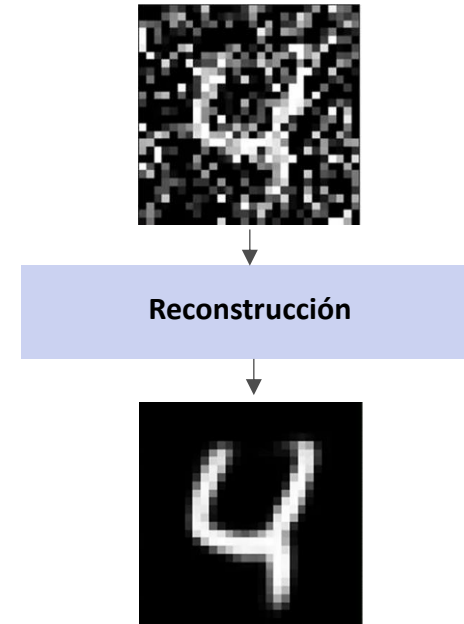
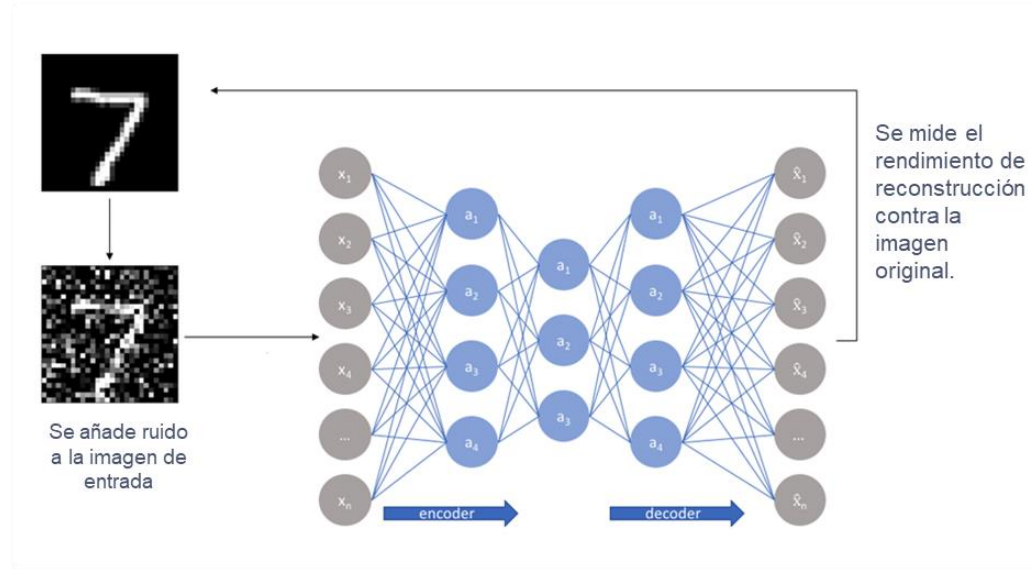
- Paso hacia adelante: para una entrada $\mathbf{x}$ se determina la salida de la red ($\mathbf{x}_{reconstruida}$)
- Paso hacia atrás: se calcula el error que comete la red y se determina la contribución de cada parámetro al error.
- Paso de actualización: se ajustan los parámetros según descenso por el gradiente.

❑ Autocodificador denoising

- Se añade ruido a la entrada, pero la salida se compara con la entrada original.



Ejemplo



Entrenamiento con BP (por ejemplo):

Entrada a la red = x

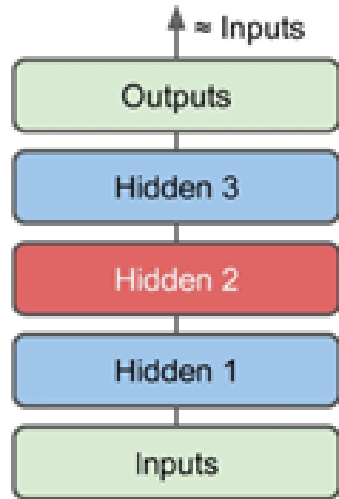
Salida de la red = $x_{reconstruida}$

Función de costo = MSE (entre x y $x_{reconstruida}$)

Funciones de activación = sigmoide, ReLU, ...

- Se aplica ruido gaussiano a la entrada $x \rightarrow x_{ruido}$
- Paso hacia adelante: para x_{ruido} se determina la salida de la red ($x_{reconstruida}$)
- Paso hacia atrás: se calcula el error que comete la red (entre x y $x_{reconstruida}$) y se determina la contribución de cada parámetro al error.
- Paso de actualización: se ajustan los parámetros según descenso por el gradiente.

❑ Autocodificador sparse



Idea: código sparse =
activaciones de las neuronas de
la capa oculta $\cong 0$

Entrada a la red = x

Salida de la red = $x_{reconstruida}$

Funciones de activación = sigmoide, ReLU, Softplus.

Función de costo = $MSE(\text{entre } x \text{ y } x_{reconstruida}) + \alpha \Omega$ Condición de dispersión (sparsity)

Por ejemplo:

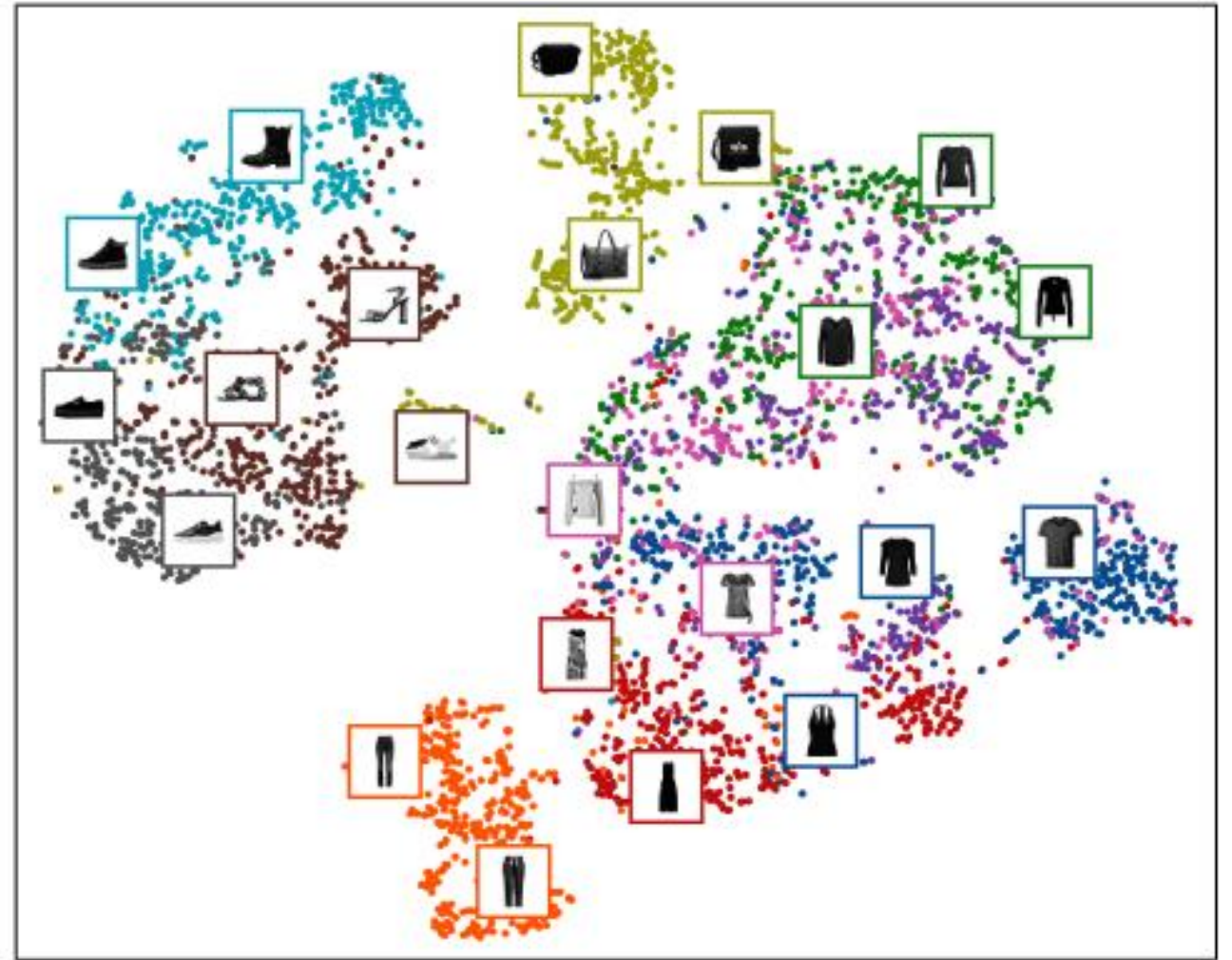
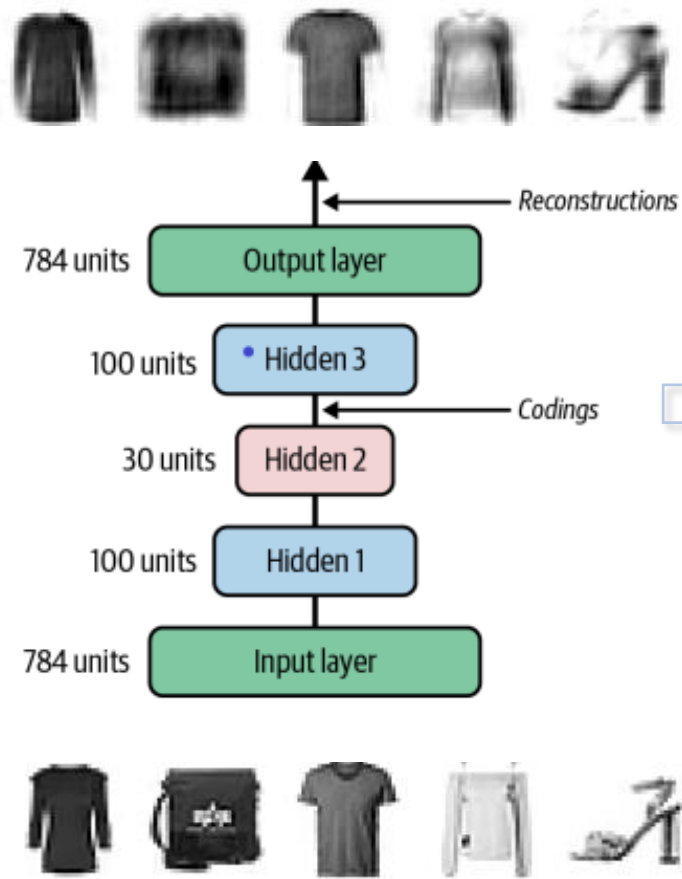
$$\text{Función de costo} = MSE(x, x_{reconstruida}) + \sum_{i=1}^{N.\text{ocultas}} |a_i|$$

Entrenamiento con BP:

- Paso hacia adelante: para una entrada x se determina la salida de la red ($x_{reconstruida}$)
- Paso hacia atrás: se calcula el error que comete la red y se determina la contribución de cada parámetro al error. Para el cálculo de los gradientes se utiliza la función de costo regularizada para producir una representación *sparse*.
- Paso de actualización: se ajustan los parámetros según descenso por el gradiente.

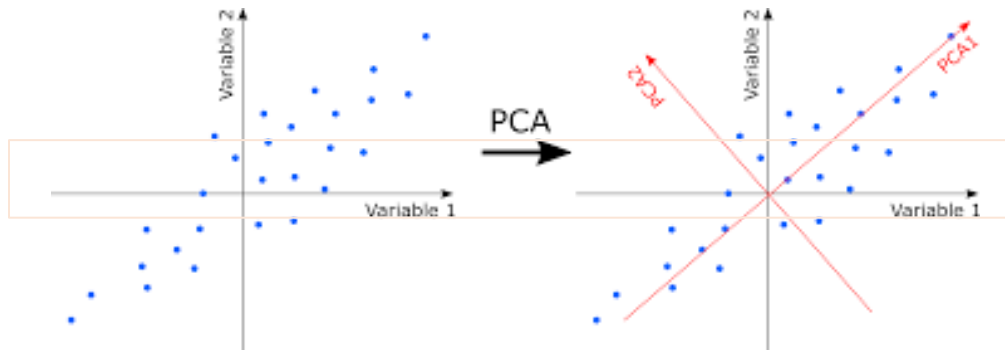
❑ Aplicaciones de los autocodificadores

- Aprendizaje de espacios latentes (extracción de características).



Conjunto de imágenes Fashion MNIST

- Reducción de la dimensionalidad



$$\text{Datos} = X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{Nd} \end{bmatrix}$$

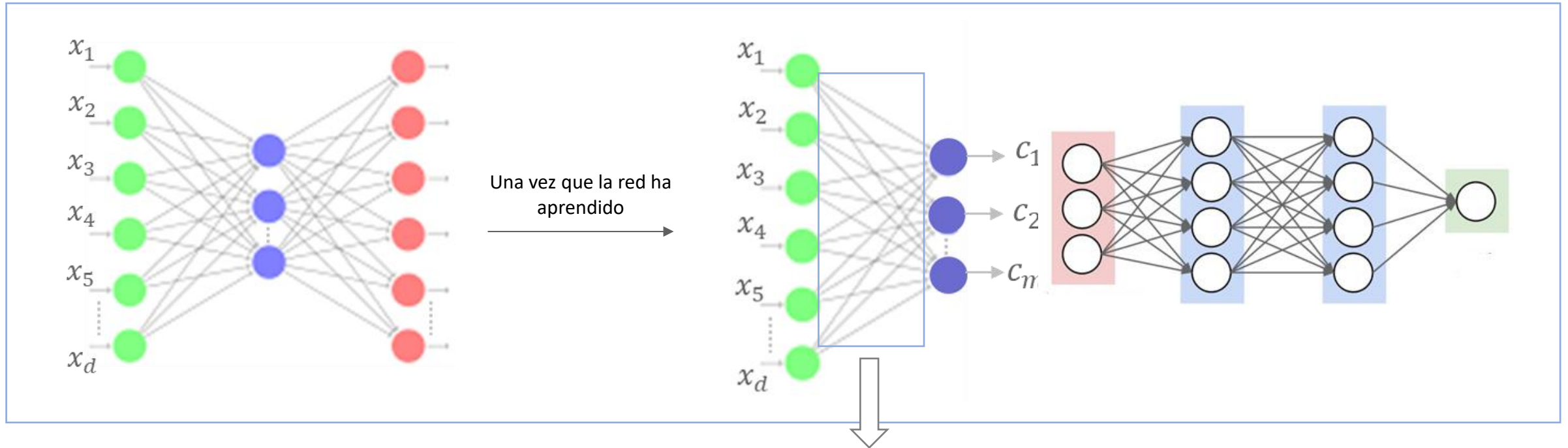
Una vez que se determinan los componentes:

Matriz de transformación
(proyección)

$$X_{\text{nueva}} = X A_m = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{Nd} \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1m} \\ a_{21} & a_{22} & \dots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{d1} & a_{d2} & \dots & a_{dm} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1m} \\ c_{21} & c_{22} & \dots & c_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ c_{N1} & c_{N2} & \dots & c_{Nm} \end{bmatrix}$$

$$X_{\text{nueva}} = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1m} \\ c_{21} & c_{22} & \dots & c_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ c_{N1} & c_{N2} & \dots & c_{Nm} \end{bmatrix}$$

Datos transformados ($m < d$). Sobre este nuevo conjunto se realiza la tarea de aprendizaje.



$$X_{nueva} = XA_m = f \left[\begin{bmatrix} x_{11} & x_{12} & \dots & x_{1d} \\ x_{21} & x_{22} & \dots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{Nd} \end{bmatrix} \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1m} \\ w_{21} & w_{22} & \dots & w_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d1} & w_{d2} & \dots & w_{dm} \end{bmatrix} \right] = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1m} \\ c_{21} & c_{22} & \dots & c_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ c_{N1} & c_{N2} & \dots & c_{Nm} \end{bmatrix}$$

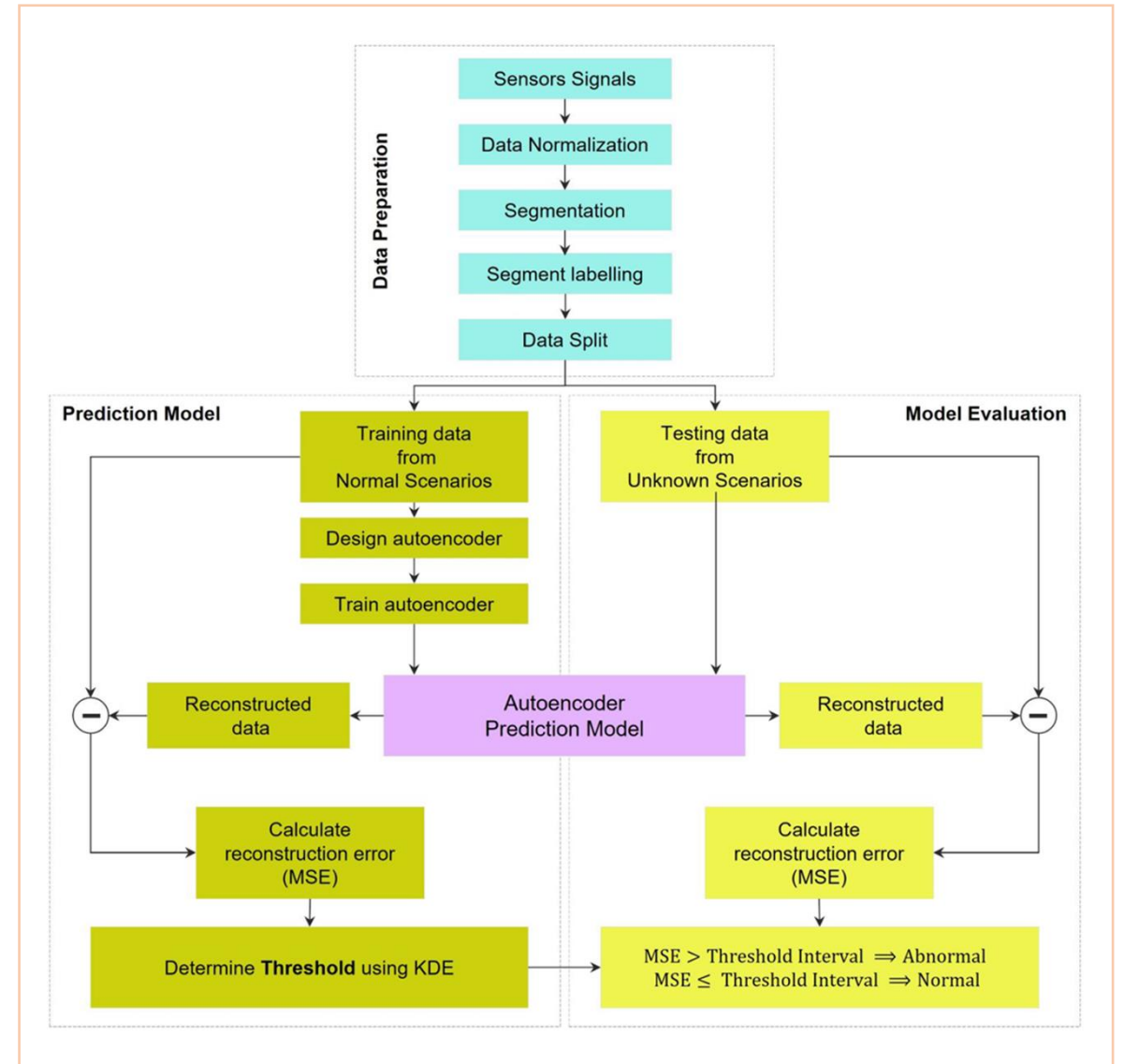
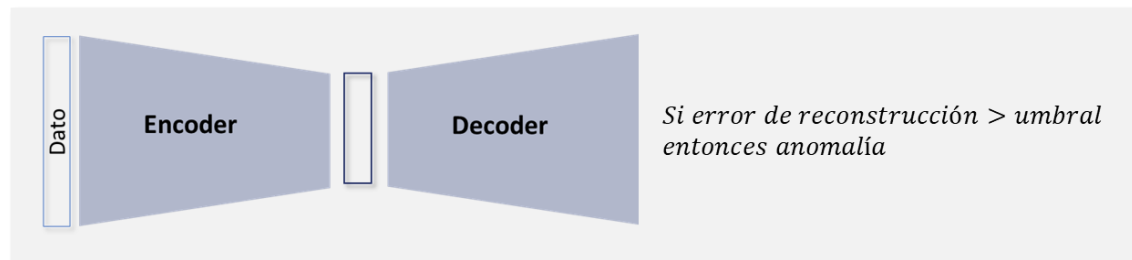
Matriz de transformación
(proyección)

Si las funciones de activación del *autoencoder* son lineales $\cong$ PCA

Pero, los autoencoders pueden realizar una reducción de la dimensionalidad no lineal

- Detección de anomalías

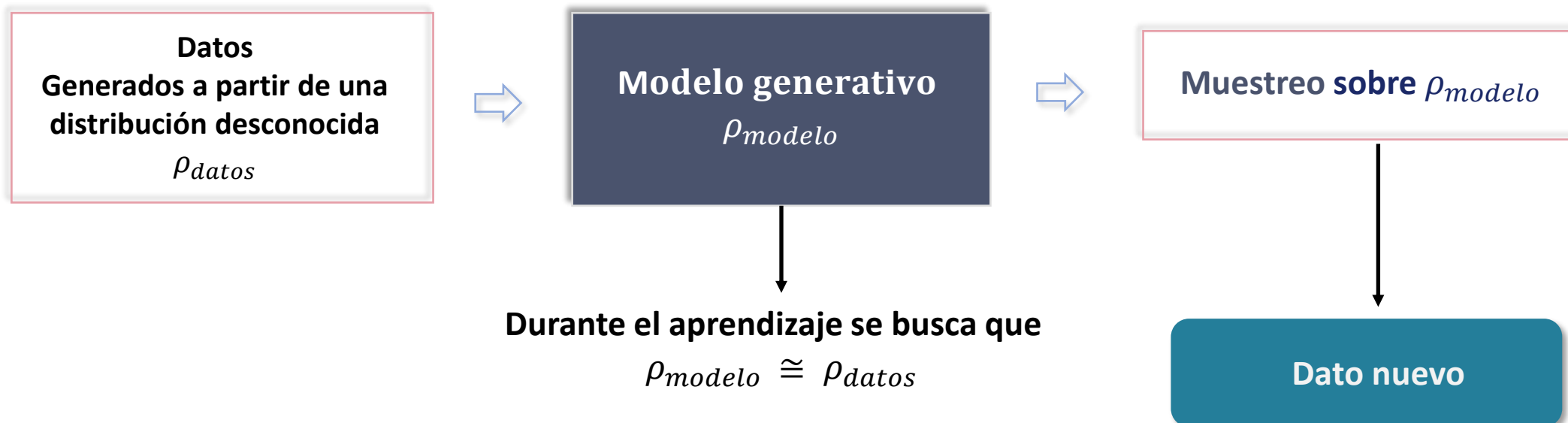
- Durante el entrenamiento solo se utilizan los datos de la clase normal, con el fin de aprender una representación latente de ellos.
- Una vez entrenado, se espera que los errores de reconstrucción de los datos normales sean muy pequeños en comparación con el error de reconstrucción de una anomalía.
- Se puede entonces fijar un umbral para realizar la detección de datos anómalos.



Modelos generativos

Un modelo generativo describe cómo se genera un conjunto de datos, en términos de un modelo probabilístico. Realizando un muestreo a partir de esta función de densidad de probabilidad se pueden generar nuevos datos.

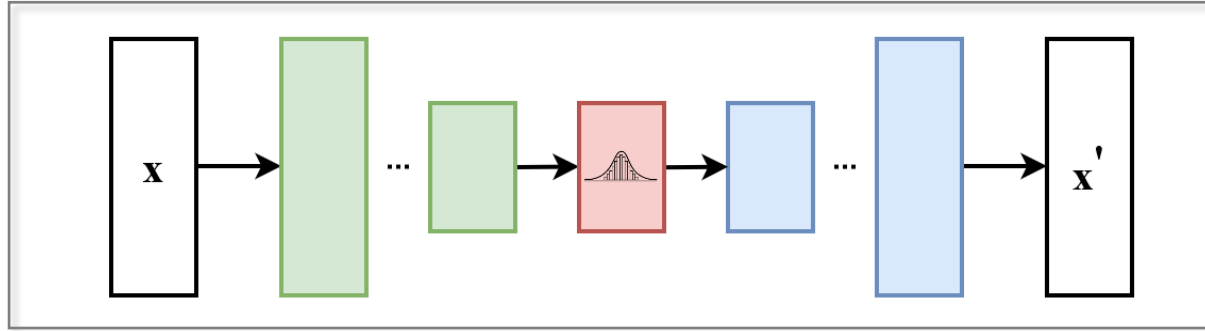
Marco general



- Los datos nuevos deben parecer como si hubiesen sido generados por ρ_{datos}
- ρ_{modelo} no debería generar datos que ya ha visto.

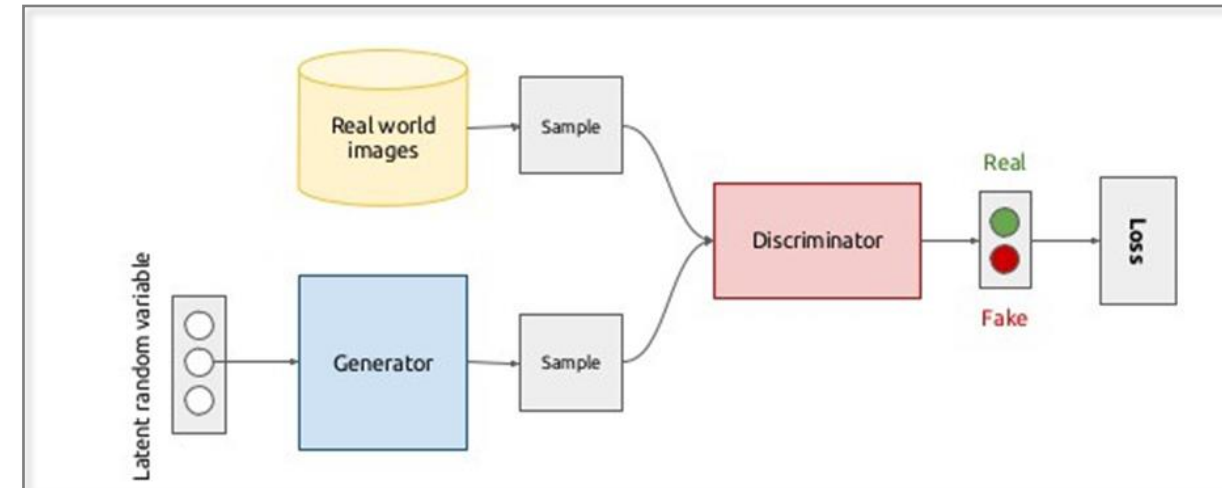
❑ Algunos modelos generativos basados en deep learning

- Autocodificador variacional (VAE).

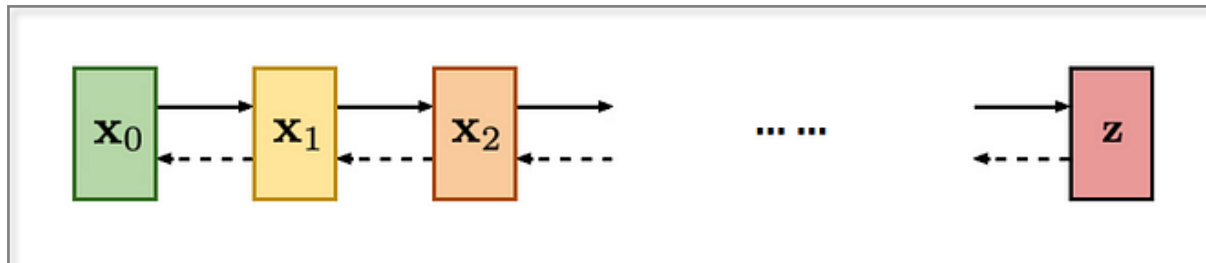


Cómo se modela la función de densidad $\rho_{\theta}(x)$ da lugar a diferentes modelos

- Redes generativas de adversarios (GAN).



- Redes de difusión.



<https://medium.com/@saibharath897/generative-adversarial-networks-gans-560c5c988128>

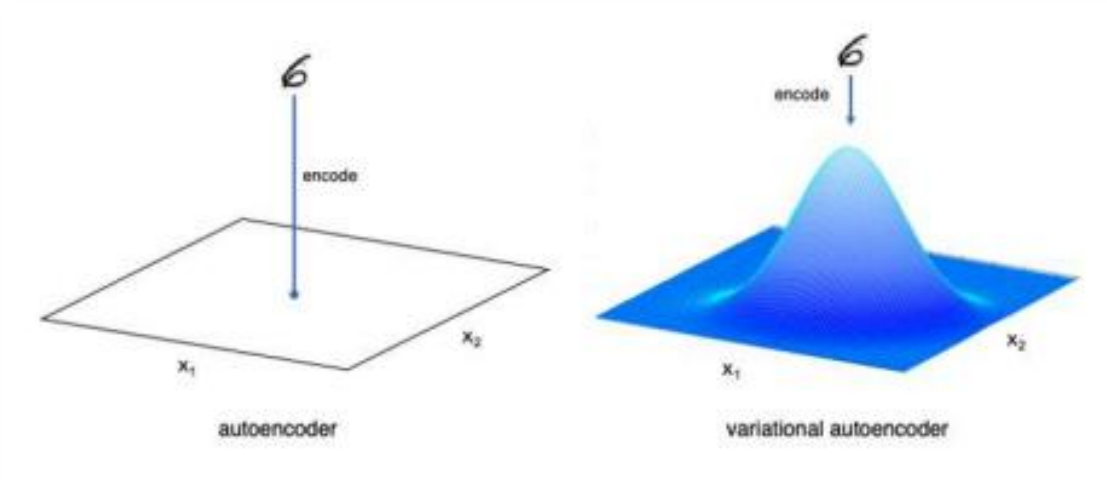
❑ El autocodificador variacional (VAE)

¿Cómo modificar el autoencoder para convertirlo en un modelo generativo?



Objetivo: muestrear el espacio latente (seleccionar un punto de manera aleatoria), pasarlo al decoder y generar un dato que parezca real.

Encoder



- Mapea a una distribución normal multivariable en torno a un punto del espacio latente.
- ¿Cómo? Toma un dato de entrada, lo codifica en un vector de medias y un vector de varianza (en la práctica se utiliza el logaritmo de la varianza).
- El VAE asume independencia entre las características del espacio latente.

Distribución normal:

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{-(x-\mu)}{-2\sigma^2}}$$

En el caso multivariable:

μ = vector de medias

Σ = matriz de covarianza

Para muestrear un punto:

$$z = \mu + \sigma \epsilon$$

ϵ = punto muestreado a partir de la distribución normal estándar

- El encoder no produce directamente un vector latente z , sino dos conjuntos de parámetros para cada ejemplo de entrada:

$$\mu(x), \sigma(x)$$

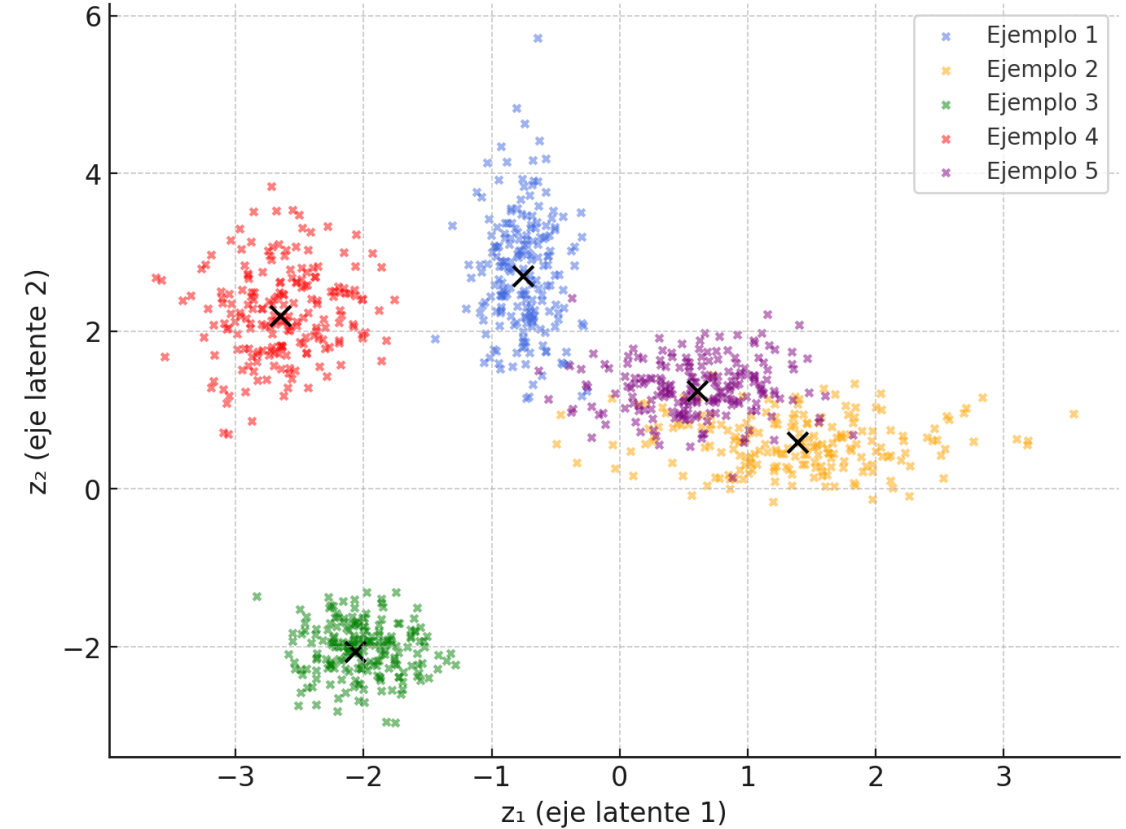
que representan la media y la desviación estándar (o varianza) de una distribución gaussiana multivariable

- A partir de esas distribuciones, se muestra un vector latente:

$$z = \mu(x) + \sigma(x)\varepsilon, \quad \varepsilon \sim N(0, I)$$

En un VAE no solo se aprenden los parámetros μ y σ , sino que también se generan representaciones latentes z al muestrear de esta distribución. Se construye así un espacio latente continuo y estructurado que el decodificador usa para generar nuevos datos.

Espacio latente 2D en un VAE (cada grupo = distribución $q(z|x)$)



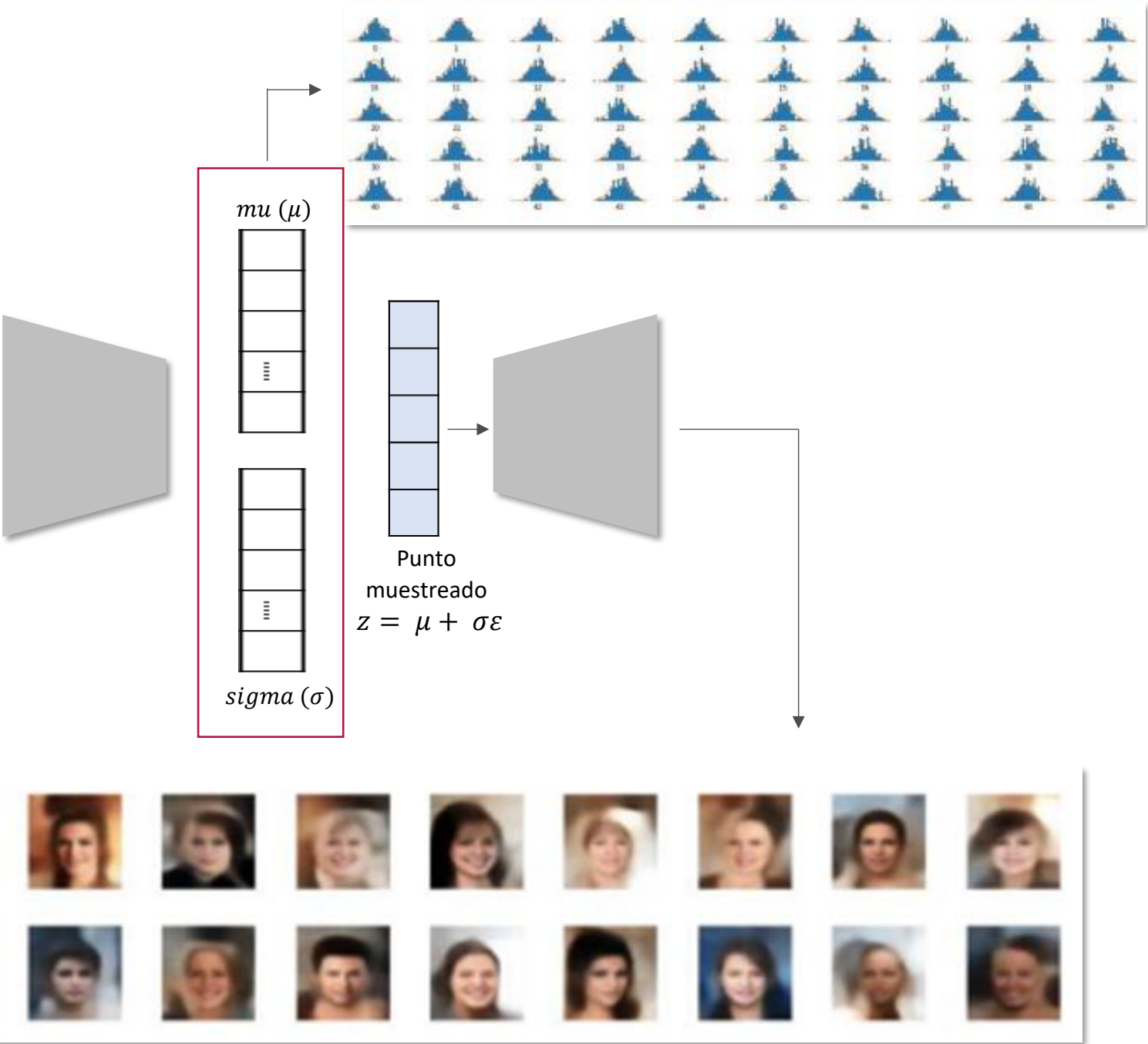
- Cada color representa la distribución $q(z | x)$ aprendida por el encoder para una muestra diferente del conjunto de datos.
- Las cruces negras ($\times$) son las medias $\mu(x)$.
- Los puntos de colores son muestras generadas con:

$$z = \mu(x) + \sigma(x)\varepsilon$$

Ejemplo: generación de imágenes



CelebFaces Attributes (CelebA) dataset.



Entrada a la red = $\mathbf{x}$

Salida de la red = $\mathbf{x}_{reconstruida}$

Funciones de activación = sigmoide, ReLU, ...

Función de costo = $MSE(\text{entre } \mathbf{x} \text{ y } \mathbf{x}_{reconstruida}) + latent\ loss$

Divergencia de Kullback–Leibler: mide cuánto una función de probabilidad difiere de otra.

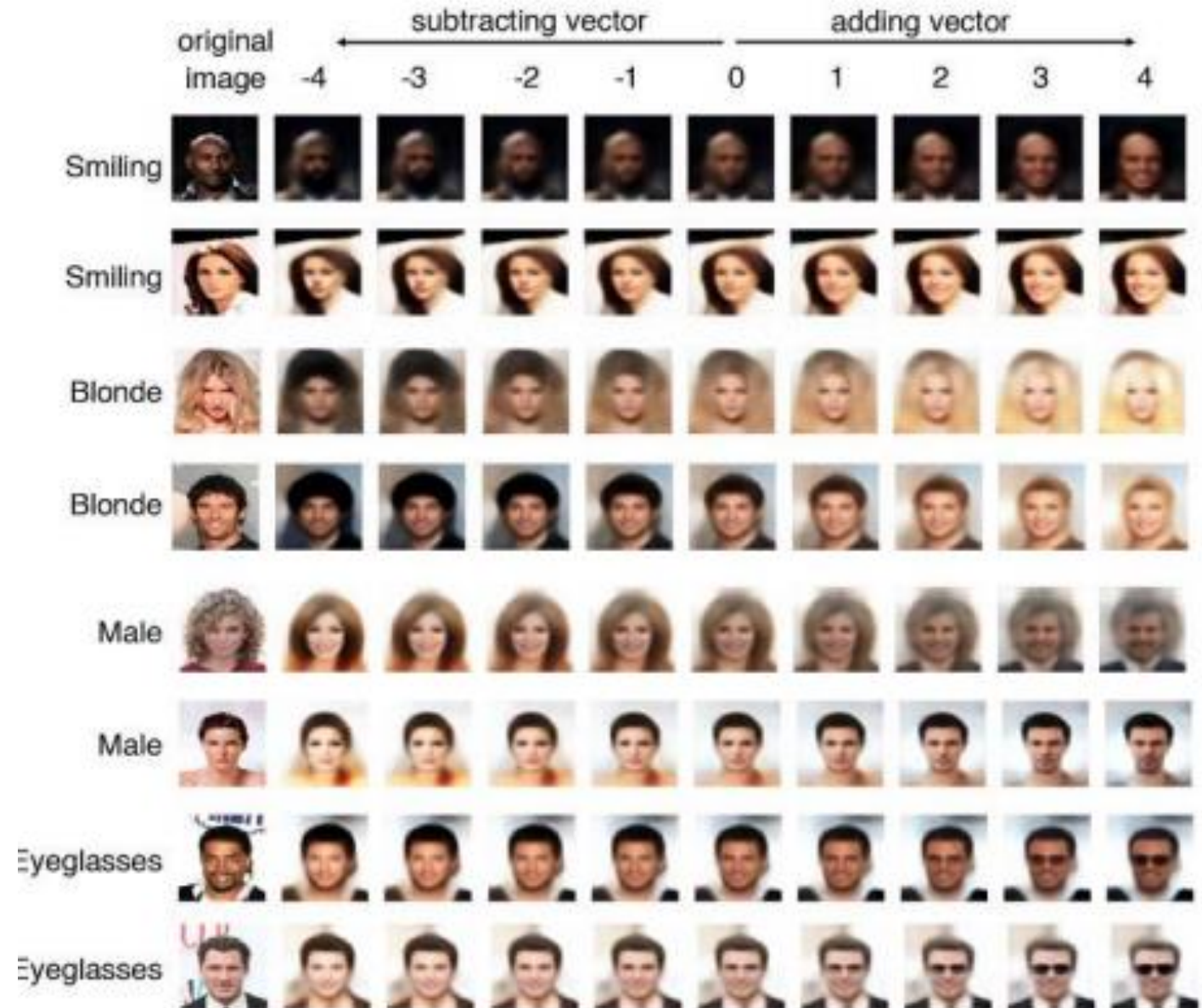
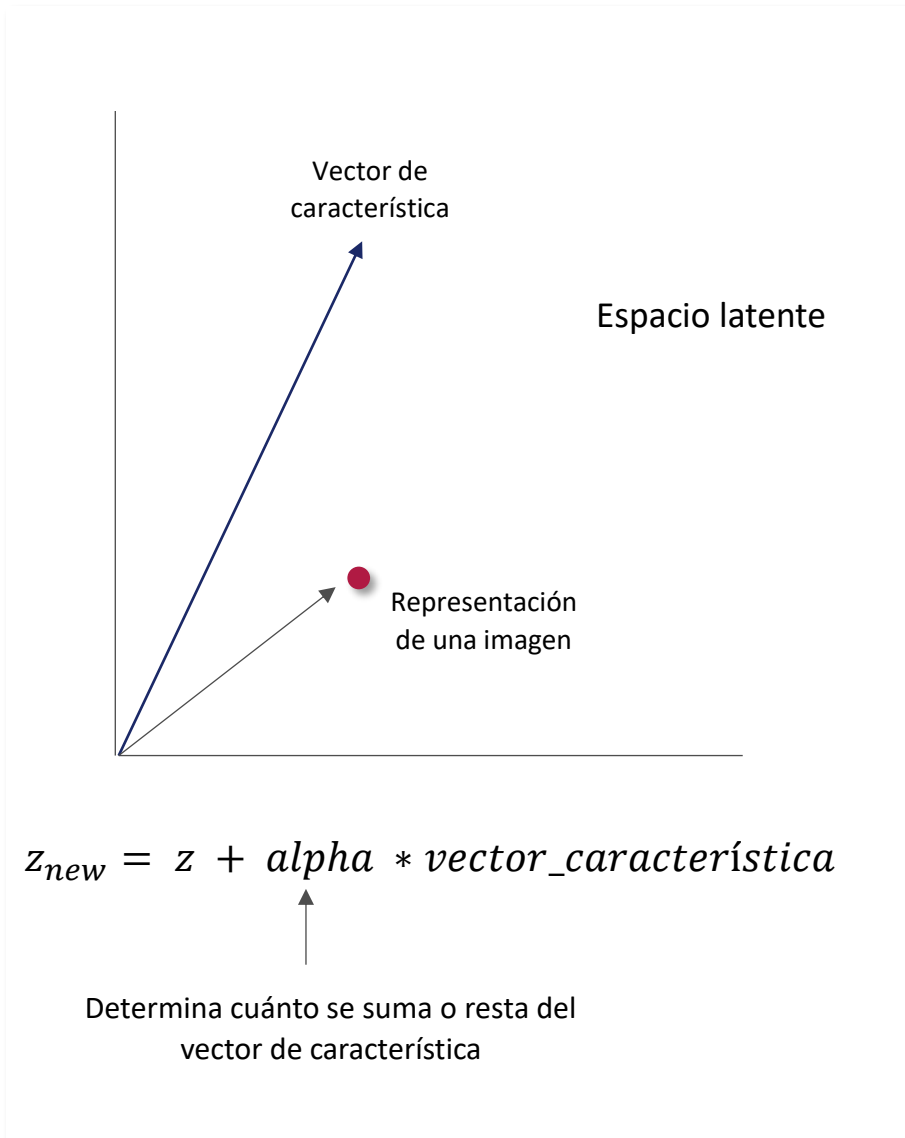
$$latent\ loss = kl_{loss} = -0.5 \sum_{i=1}^K [1 + \log(\sigma_i^2) - \exp(\log(\sigma_i^2) - \mu_i^2)]$$

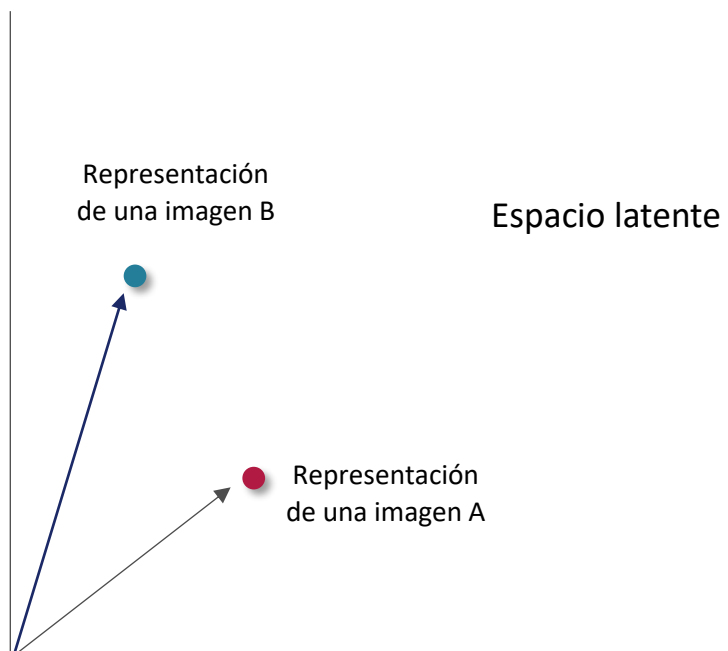
Entrenamiento:

- **Paso hacia adelante:** para una entrada $\mathbf{x}$ se determina la salida de la red ($\mathbf{x}_{reconstruida}$) ¿Cómo? La entrada se codifica como una distribución sobre el espacio latente. Luego, se muestrea un punto del espacio latente a partir de esa distribución. Por último, se decodifica el punto muestreado y se calcula el error de reconstrucción
- **Paso hacia atrás:** se calcula el error que comete la red y se determina la contribución de cada parámetro al error. Para el cálculo de los gradientes se utiliza la función de costo con el término que mide el error sobre el espacio latente.
- **Paso de actualización:** se ajustan los parámetros según descenso por el gradiente.

El término de divergencia KL penaliza la red al codificar observaciones a las variables μ y $\log_var$ que difieren significativamente de los parámetros de una distribución normal estándar: $\mu = 0$ y $\log_var = 0$.

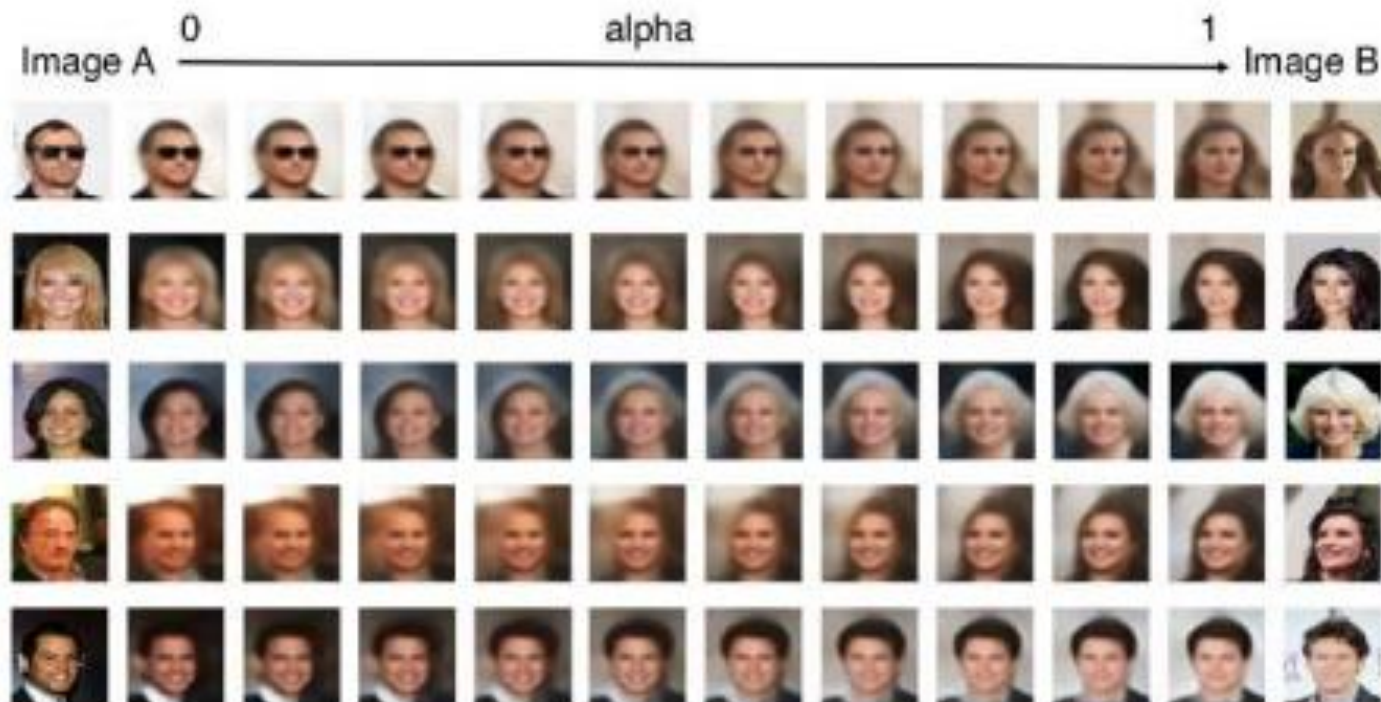
❑ Operaciones en el espacio latente





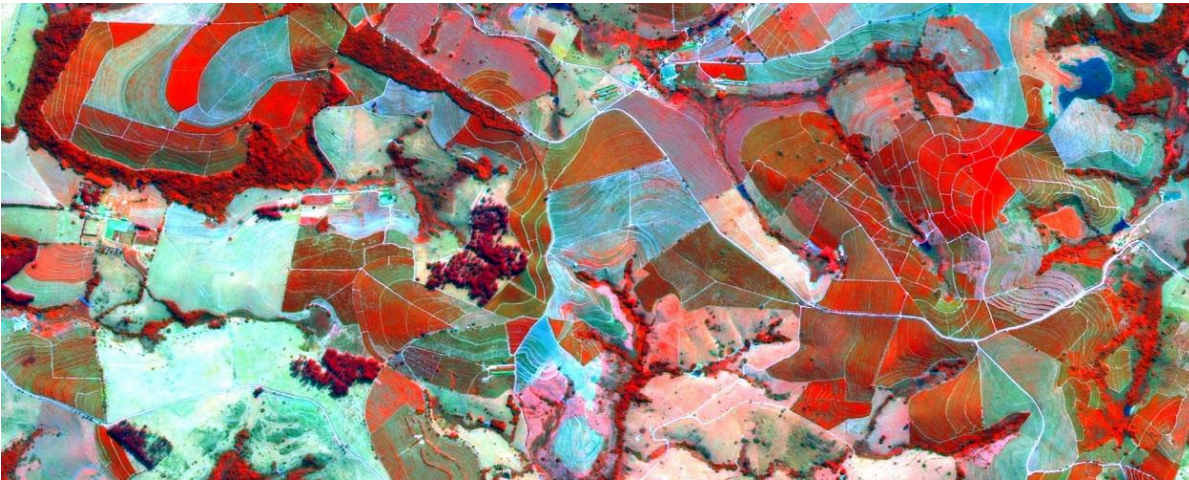
$$z_{new} = z_A * (1 - \alpha) + z_B * \alpha$$

↑
Determina cuánto se suma o resta del
vector de característica

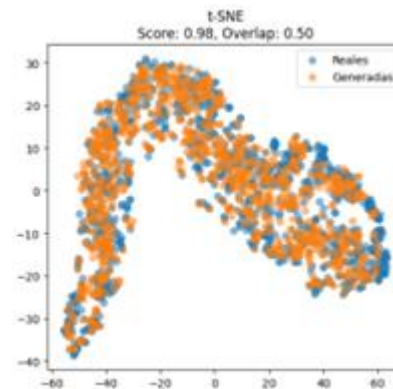


Proyecto. Uso de técnicas de deep learning para la generación y análisis de datos geoespaciales: un caso de estudio en la identificación de cultivos sobre imágenes satelitales.

El objetivo de este proyecto es aplicar técnicas de deep learning para optimizar la clasificación de cultivos a partir de imágenes satelitales. Se emplearán modelos generativos para la aumentación de *patches* de imágenes, con el fin de ampliar y diversificar el conjunto de datos disponibles para entrenar redes neuronales especializadas en el procesamiento de imágenes. Este enfoque busca no solo mejorar la precisión de los modelos de clasificación, sino también enfrentar los desafíos derivados de la escasez de datos etiquetados y la variabilidad inherente a las imágenes satelitales.

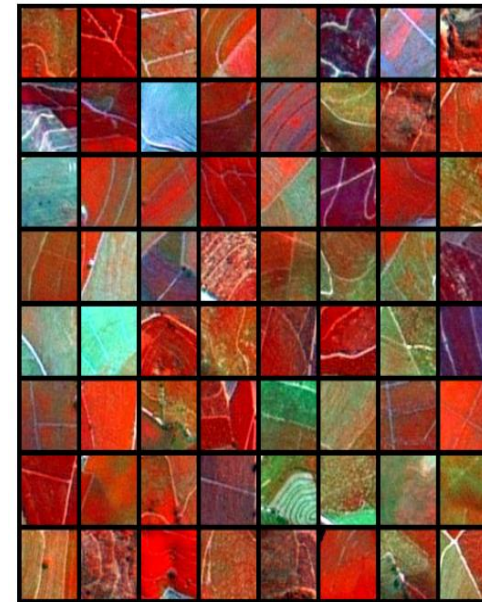


Brazilian Coffee Scenes Dataset

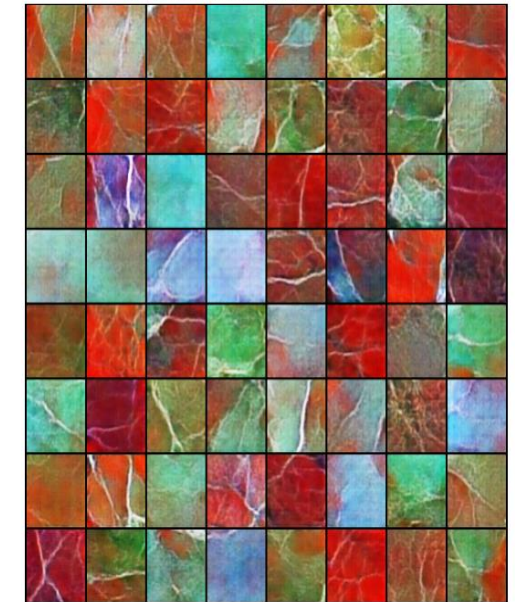


Con autocodificadores variacionales

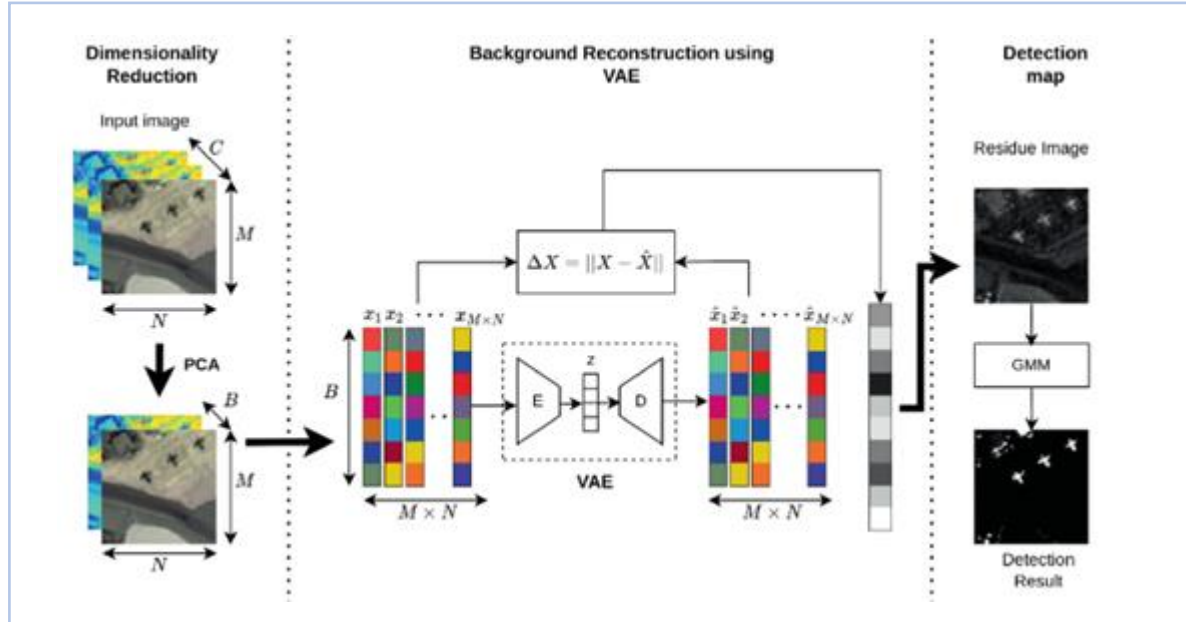
Imágenes Reales (del último lote)



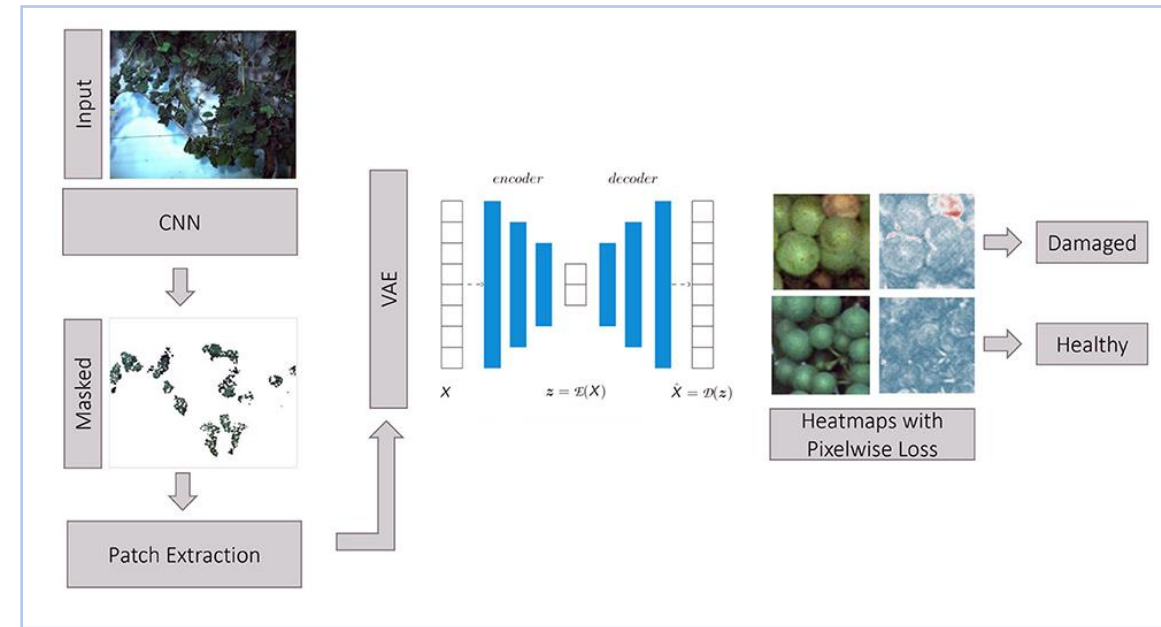
Imágenes Generadas (Última Época)



- Detección de anomalías



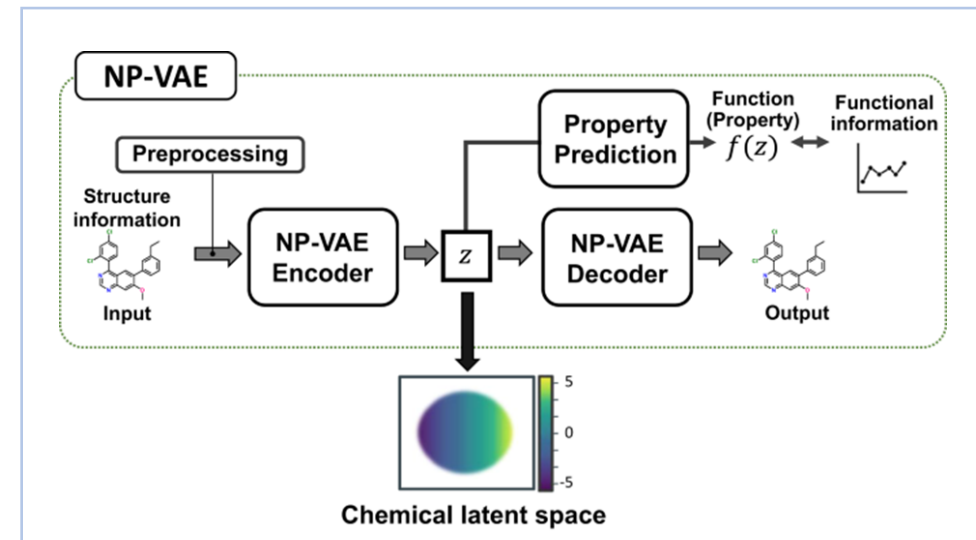
VAE-AD: Unsupervised Variational Autoencoder for Anomaly Detection in Hyperspectral Images (https://link.springer.com/chapter/10.1007/978-981-99-1648-1_11)



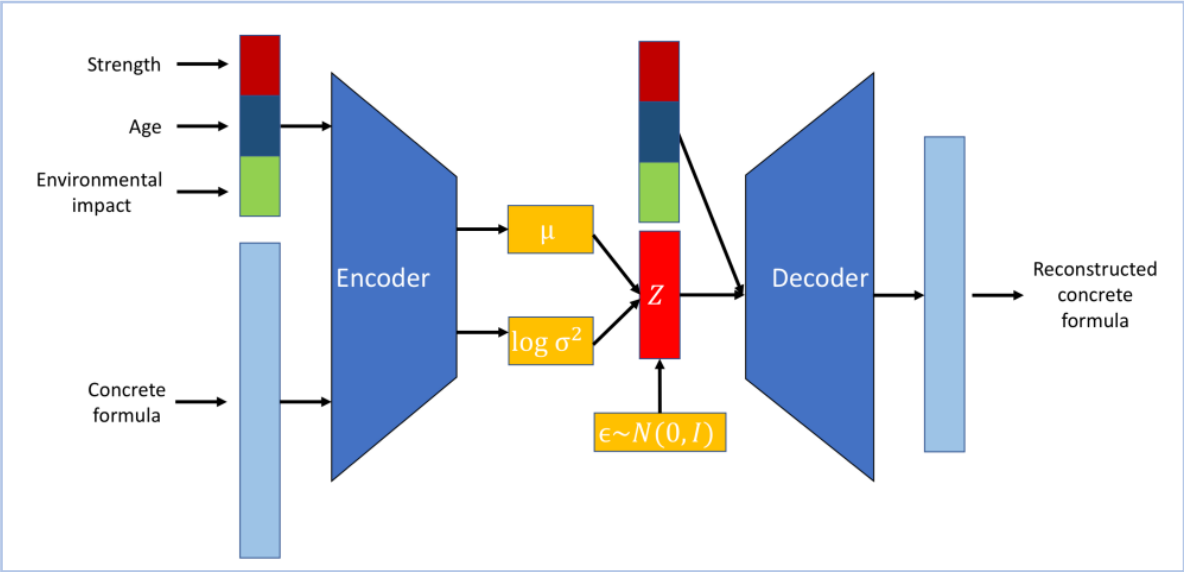
Detection of Anomalous Grapevine Berries Using Variational Autoencoders (<https://www.frontiersin.org/articles/10.3389/fpls.2022.729097/full>)

- Construcción y manipulación de espacios latentes

Variational autoencoder-based chemical latent space for large molecular structures with 3D complexity (<https://www.nature.com/articles/s42004-023-01054-6>)

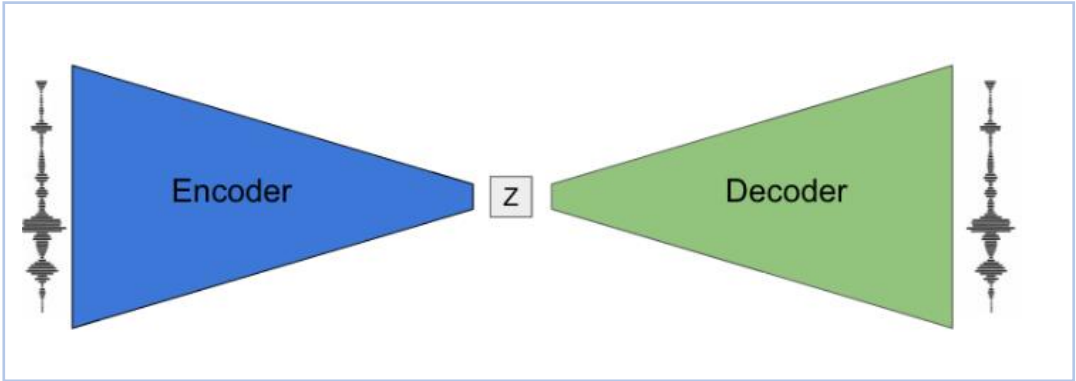


■ VAE condicional



Accelerated Design and Deployment of Low-Carbon Concrete for Data Centers
(<https://arxiv.org/pdf/2204.05397>)

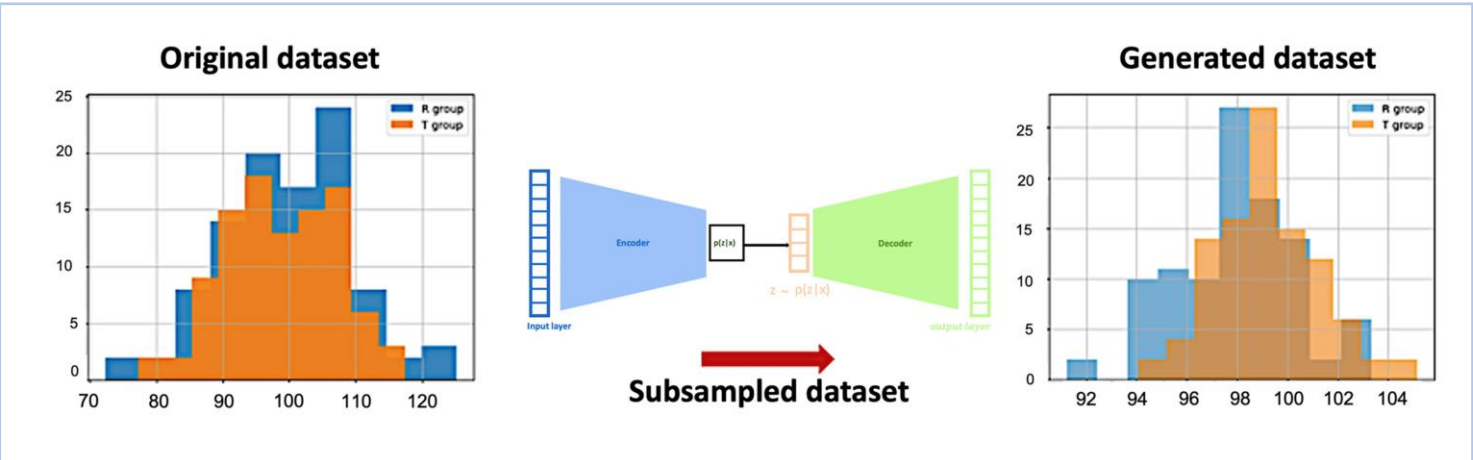
■ Generación de música



MusicVAE: Creating a palette for musical scores with machine learning
(<https://magenta.tensorflow.org/music-vae>).

Magenta (<https://magenta.tensorflow.org/>)

■ Aumentación de datos



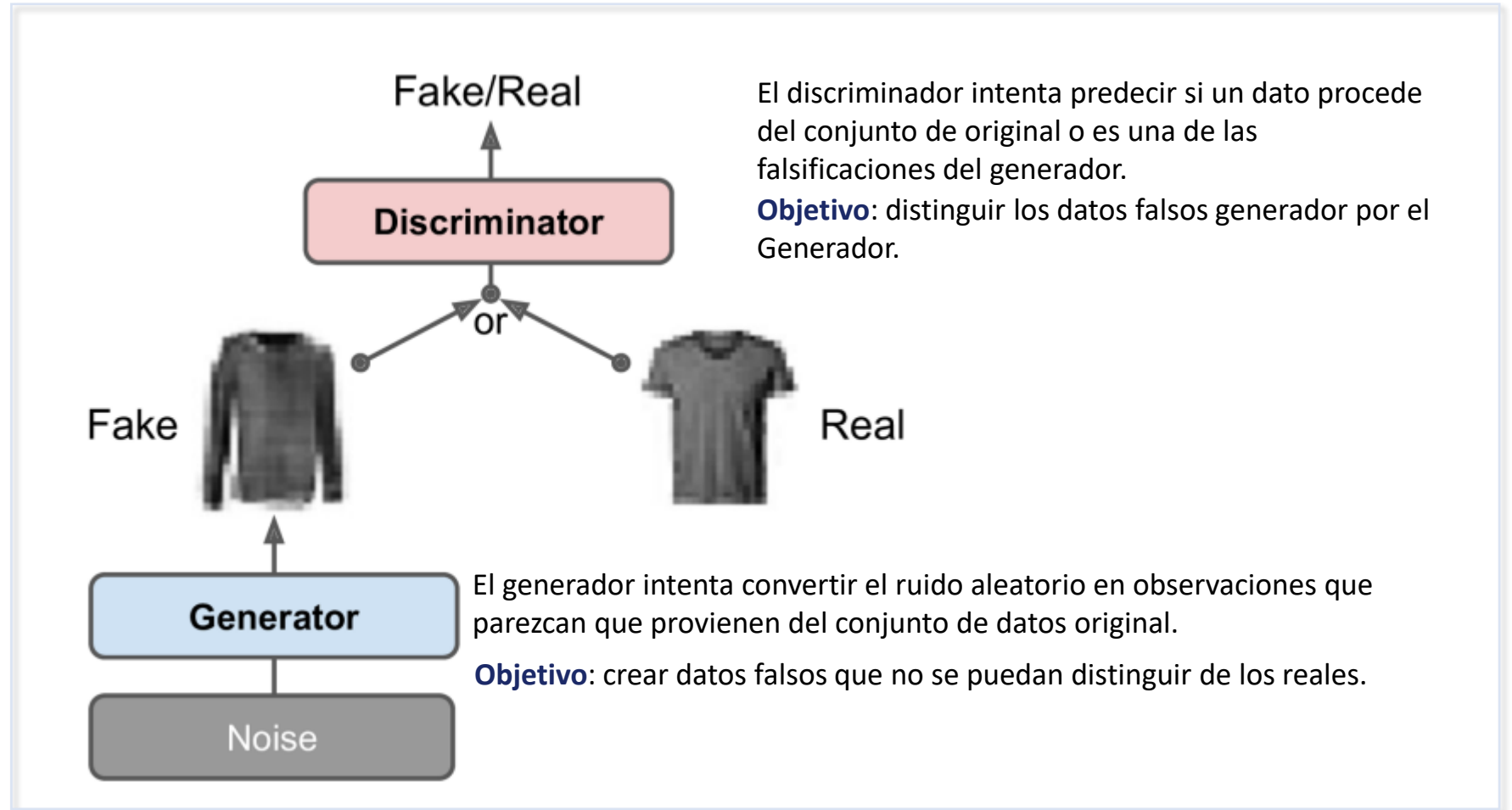
Variational Autoencoders for Data Augmentation in Clinical Studies (<https://www.mdpi.com/2076-3417/13/15/8793>)

Las redes generativas de adversarios

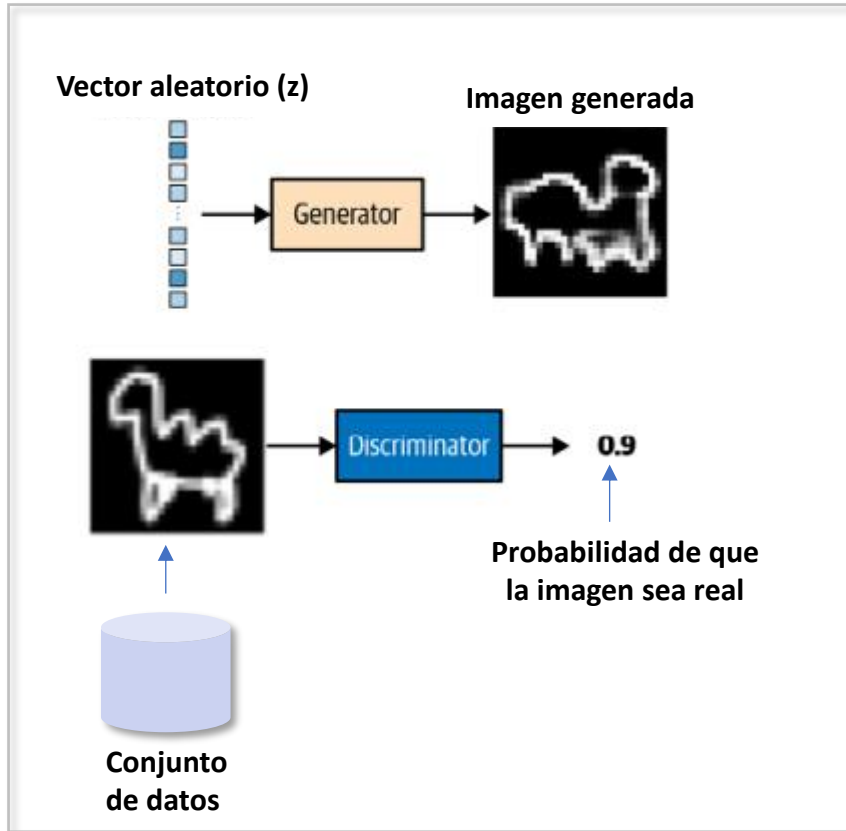
Las **redes generativas de adversarios** consisten de dos modelos: uno (el Generador) entrenado para generar datos falsos, y el otro (el Discriminador) entrenado para discernir los datos falsos de los ejemplos reales.

¿Por qué generativo? Se generan nuevos datos.

¿Por qué adversarial? Juego entre adversarios (entre dos modelos).



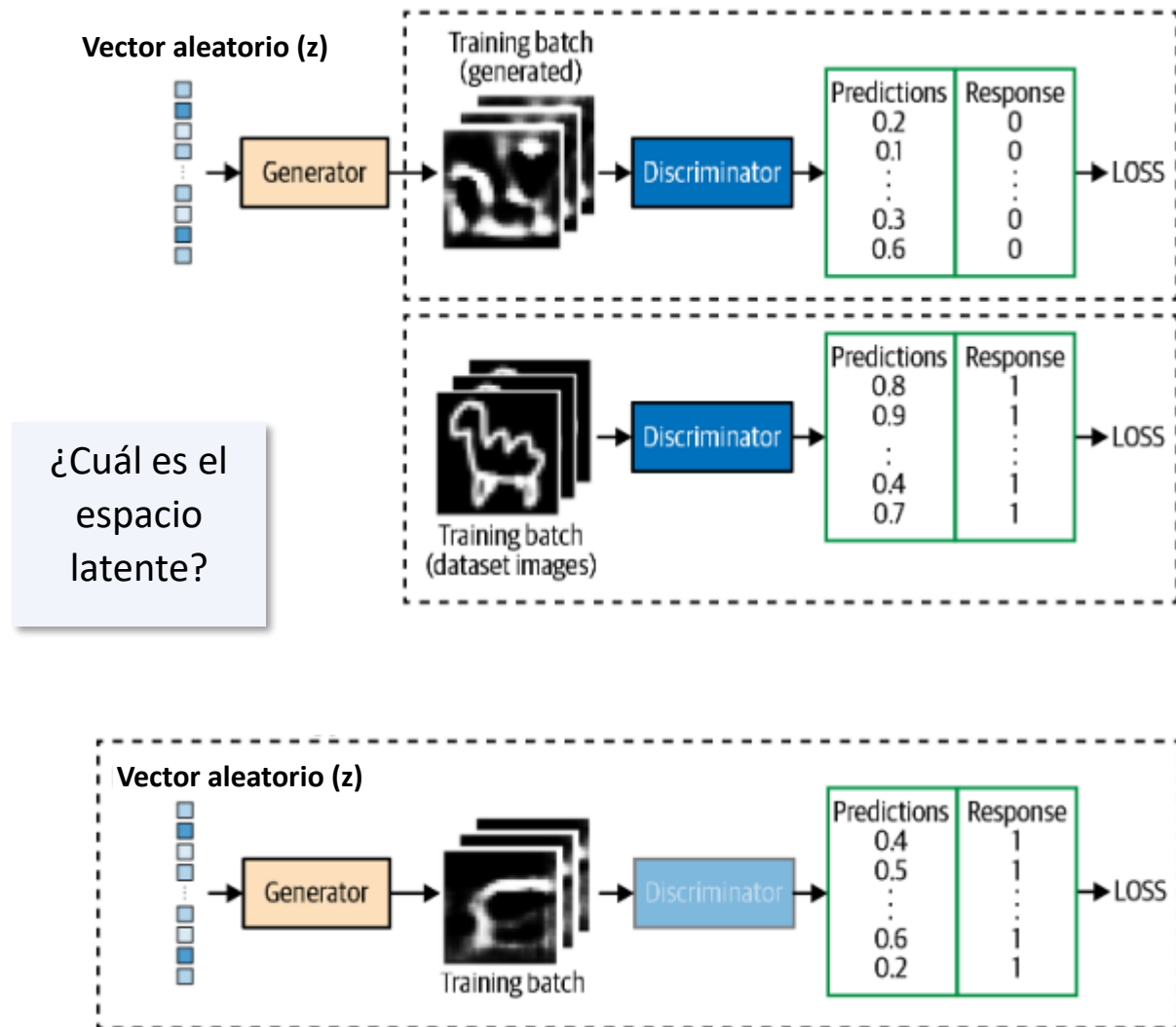
❑ Arquitectura básica



- Conjunto de datos: lo que se quiere que el Generador aprenda a generar.
- Vector aleatorio: entrada (z) al Generador. Generalmente, a partir de una distribución normal estándar.
- Generador: a partir del vector de números aleatorios (z) produce un ejemplo falso (x^*). Su objetivo es hacer que los ejemplos falsos sean indistinguibles de los ejemplos reales del conjunto de entrenamiento.
- Discriminador: toma como entrada un ejemplo real(x) procedente del conjunto de entrenamiento o un ejemplo falso (x^*) producido por el Generador. Para cada ejemplo, el discriminador determina la probabilidad de que el ejemplo sea real.
- Entrenamiento: ajusta iterativamente el Discriminador y el Generador con base en el error de clasificación.

	Generador	Discriminador
Entrada	Vector de números aleatorios	Datos reales + datos falsos (generados por el Generador)
Salida	Datos falsos tan realistas como sea posible	Predicción: probabilidad de que el dato de entrada sea real.

Entrenamiento



En cada iteración:

1. Entrenar el Discriminador:

Batch: datos reales + datos falsos generados por el Generador (1: datos reales, 0: datos falsos).

Función de costo: cross-entropy (clasificador binario).

Entrenado con backpropagation.

Solo se actualizan los pesos del Discriminador con base en el error. En este paso los pesos del Generador permanecen fijos.

2. Entrenar el Generador:

Batch: solo datos falsos (etiqueta = 1).

Función de costo: cross-entropy

Entrenado con backpropagation.

Solo se actualizan los pesos del Generador con base en el error. En este paso los pesos del Discriminador permanecen fijos.

Ejemplo: conjunto de datos MNIST

```
codings_size = 30

generator = keras.models.Sequential([
    keras.layers.Dense(100, activation="selu", input_shape=
[codings_size]),
    keras.layers.Dense(150, activation="selu"),
    keras.layers.Dense(28 * 28, activation="sigmoid"),
    keras.layers.Reshape([28, 28])
])

discriminator = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(150, activation="selu"),
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(1, activation="sigmoid")
])

gan = keras.models.Sequential([generator, discriminator])

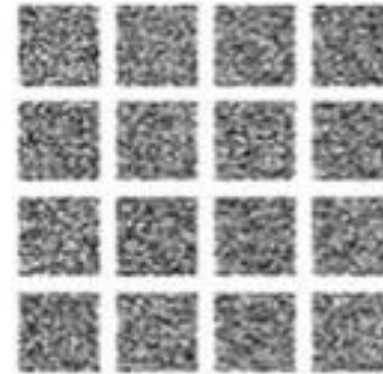
discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop")
discriminator.trainable = False
gan.compile(loss="binary_crossentropy", optimizer="rmsprop")

def train_gan(gan, dataset, batch_size, codings_size, n_epochs=50):
    generator, discriminator = gan.layers
    for epoch in range(n_epochs):
        for X_batch in dataset:
            # phase 1 - training the discriminator
            noise = tf.random.normal(shape=[batch_size, codings_size])
            generated_images = generator(noise)
            X_fake_and_real = tf.concat([generated_images, X_batch],
axis=0)

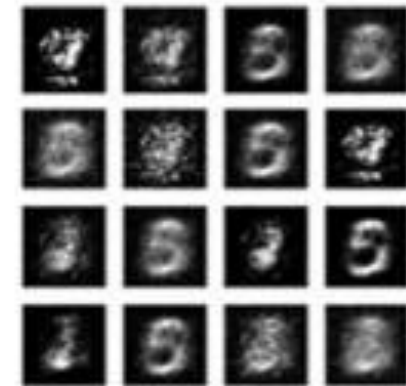
            y1 = tf.constant([[0.] * batch_size + [[1.]] * batch_size)
            discriminator.trainable = True
            discriminator.train_on_batch(X_fake_and_real, y1)
            # phase 2 - training the generator
            noise = tf.random.normal(shape=[batch_size, codings_size])
            y2 = tf.constant([[1.]] * batch_size)
            discriminator.trainable = False
            gan.train_on_batch(noise, y2)

train_gan(gan, dataset, batch_size, codings_size)
```

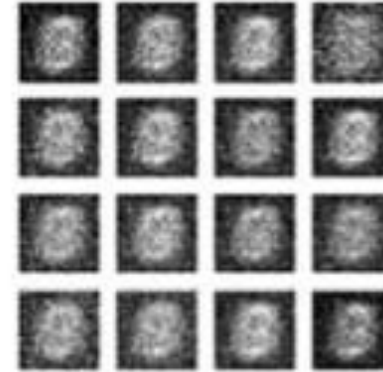
Época 1



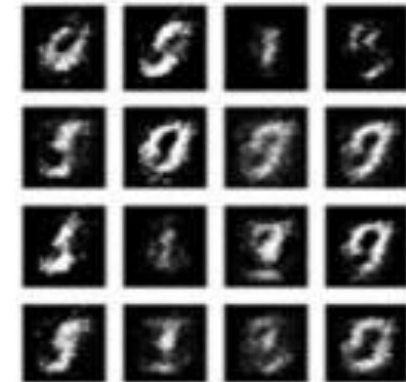
Época 3



Época 2

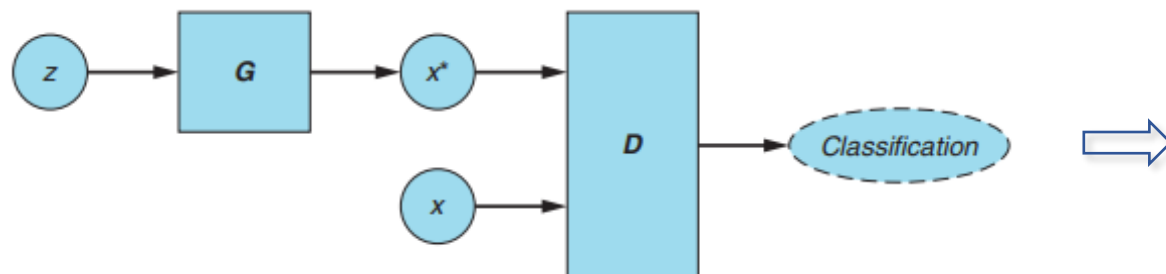


Época 4



Una vez entrenada





		Salida del Discriminador	
		Real (1)	Falso (0)
Actual	Real (x)	VP	FN
	Falso (x^*)	FP	VN

Con base en la matriz de confusión

¿Qué trata de hacer el Discriminador? ¿Y el Generador?

¿Cuándo se alcanza el punto de equilibrio?

El GAN alcanza el equilibrio de Nash cuando se cumplen las siguientes condiciones:

- El generador produce ejemplos falsos que no se distinguen de los datos reales en el conjunto de entrenamiento.
- El Discriminador puede, en el mejor de los casos, adivinar al azar si un ejemplo concreto es real o falso (es decir, adivinar al 50% si un ejemplo es real).

Juego suma cero (*zero-sum*):
la ganancia o pérdida de un jugador se equilibra con la ganancia o pérdida del otro jugador.

Equilibrio de Nash

¿Problemas?

- Entrenamiento inestable.
- Colapso de modo (*mode collapse*).
- Dificultad para monitorear la función de costo,

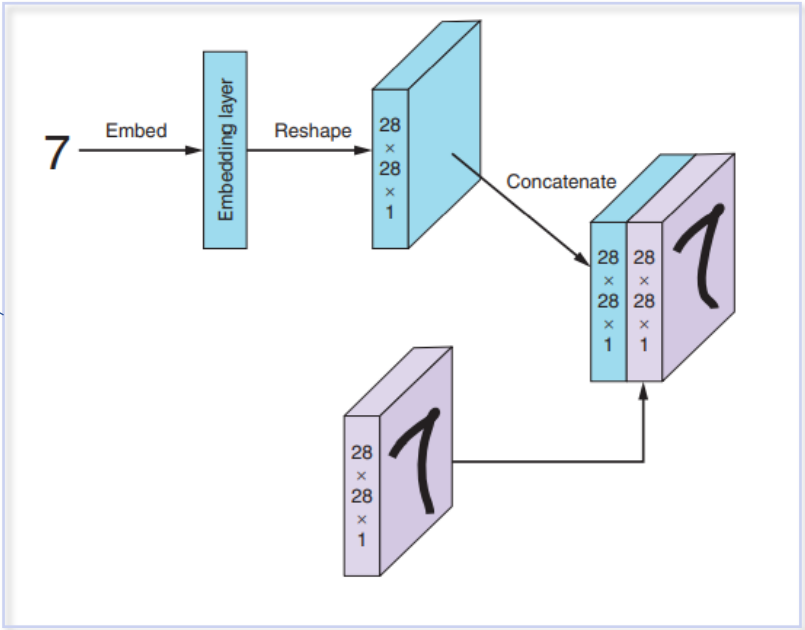
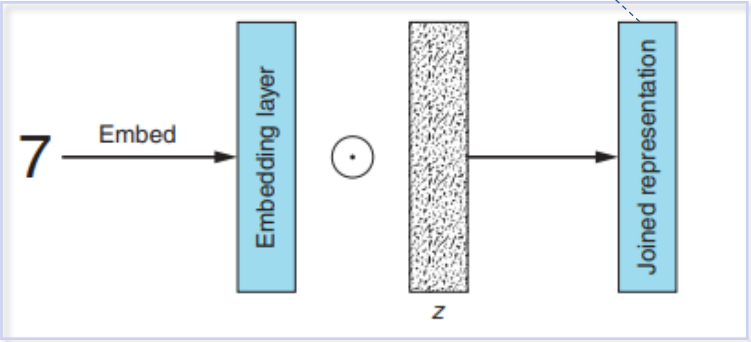
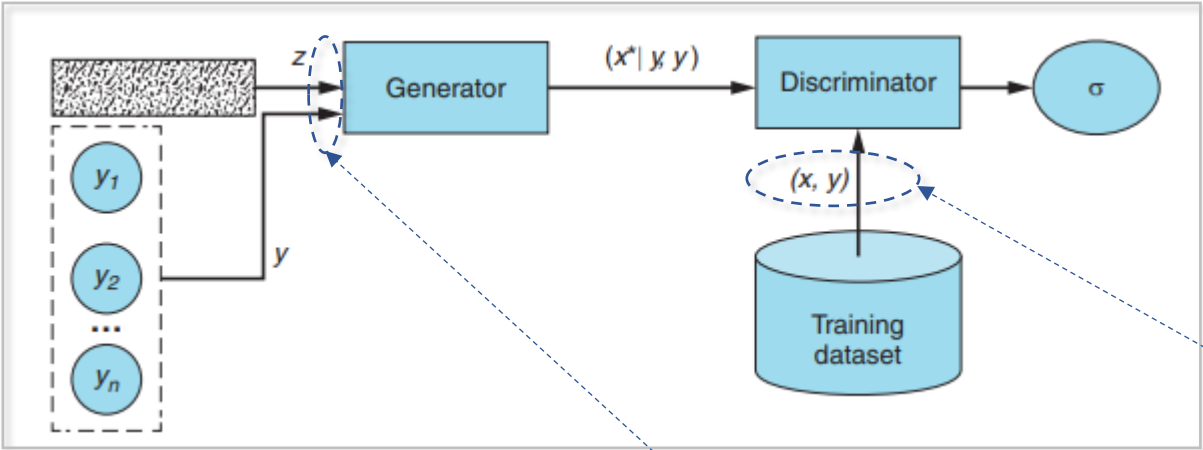
Por ejemplo,

- Wasserstein GAN. Utiliza otra función de costo: *wasserstein loss*.
- Wasserstein GAN– Gradient Penalty (WGAN-GP). Utiliza una función de costo con penalización.

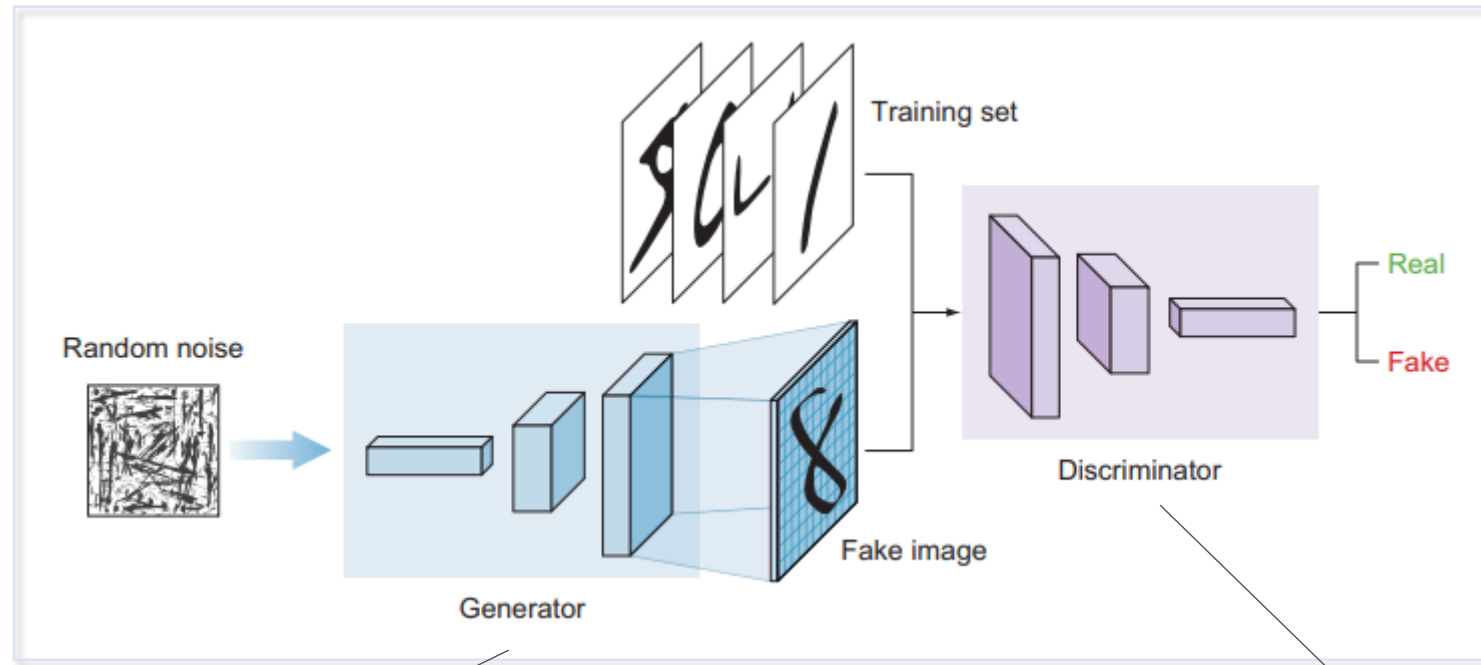
❑ Conditional GAN (CGAN)

El Generador y Discriminador se condicionan durante el entrenamiento utilizando información adicional (etiqueta de clase, conjunto de etiquetas, descripción textual,...)

	Generador	Discriminador
Entrada	Vector de números aleatorios + etiqueta = (z, y)	Pares de datos reales (x, y) + pares de datos falsos $((x^* y), y)$
Salida	Pares de datos falsos tan realistas como sea posible = $(x^* y)$	Probabilidad de que el dato de entrada sea real.

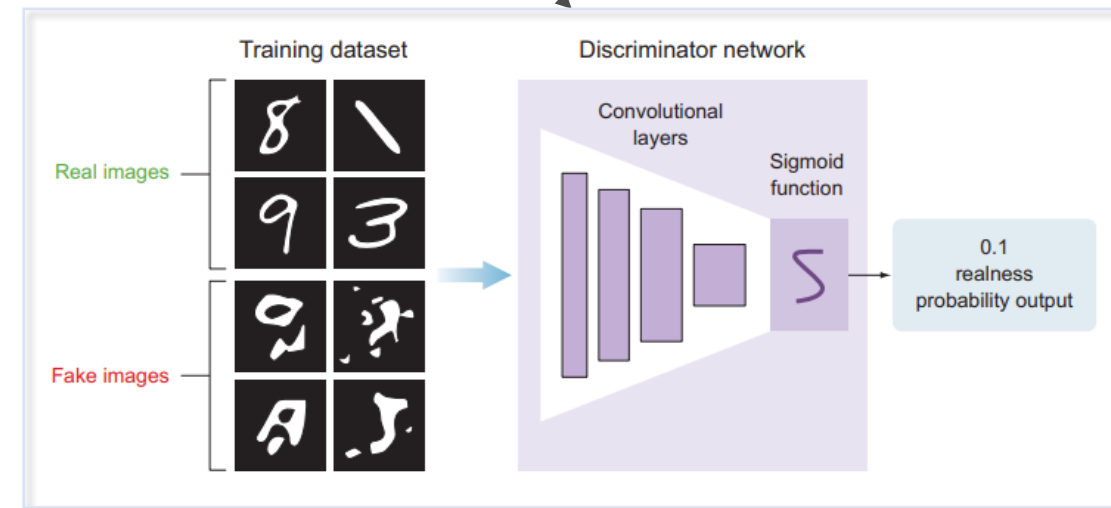
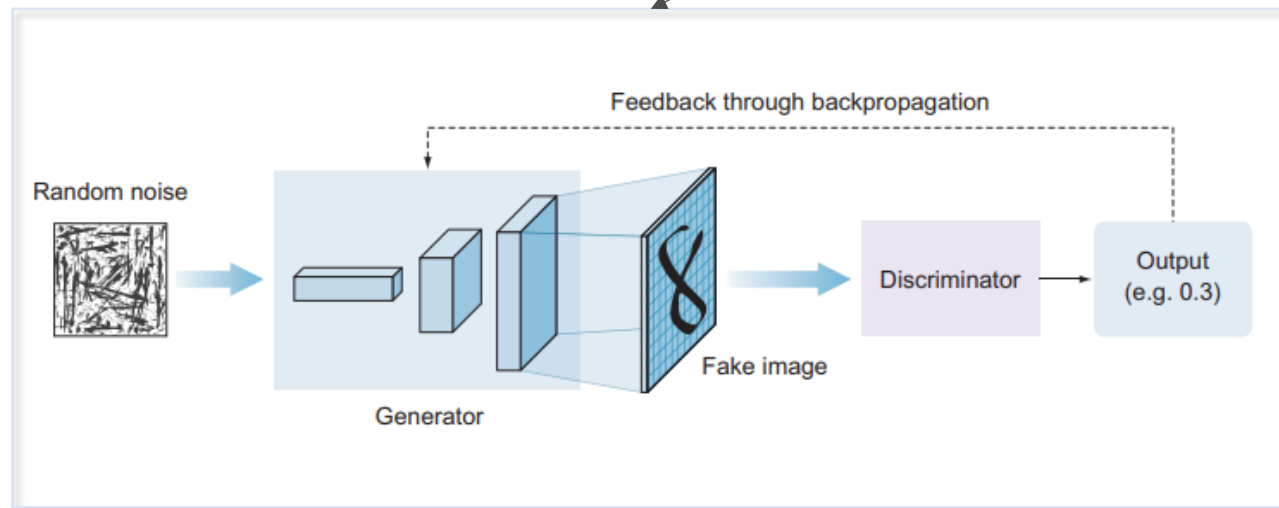


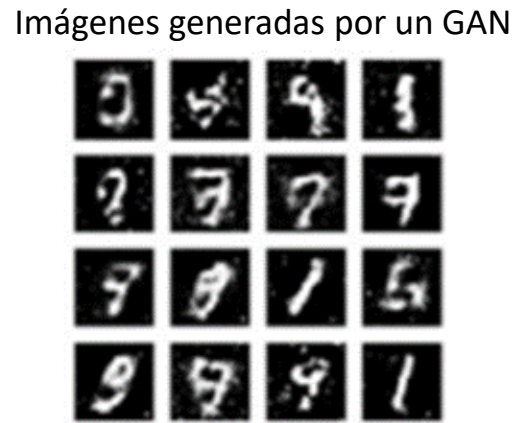
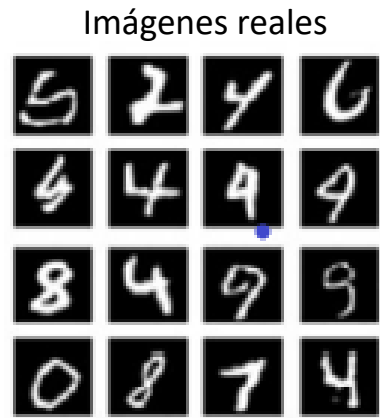
❑ Deep Convolutional GAN (DCGAN)



Entrenamiento

Entrenamiento





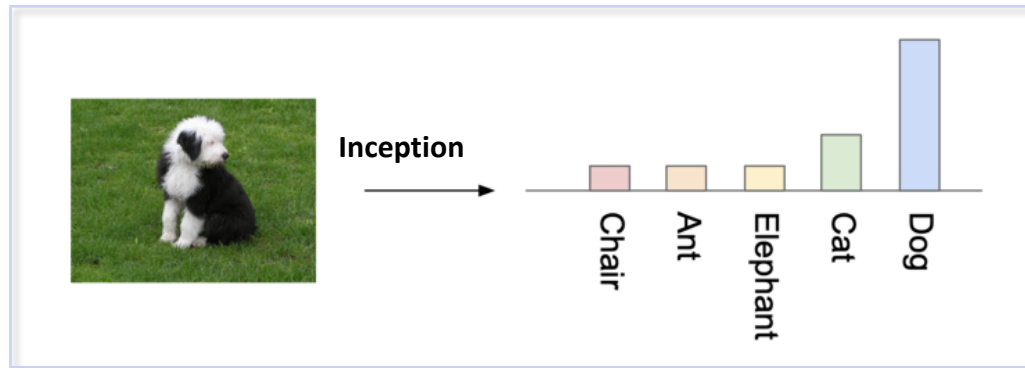
❑ ¿Cómo evaluar?

- Tanto el modelo generador como el discriminador se entrenan juntos para mantener un equilibrio.
- No hay forma de evaluar objetivamente el progreso del entrenamiento y la calidad del modelo a partir de la pérdida por sí sola.
- ¿Esto significa que los modelos deben evaluarse mediante la inspección manual de las imágenes generadas?

❑ ¿Cuáles métricas utilizar para evaluar la calidad de los datos generados?

- ¿Qué medir?
- Las imágenes generadas parecen reales.
 - El conjunto de imágenes contempla todos los objetos del conjunto de datos (*diversidad*).

Por ejemplo: score Inception (*Inception score*). Para GAN.



Probabilidad condicional (mide la calidad)

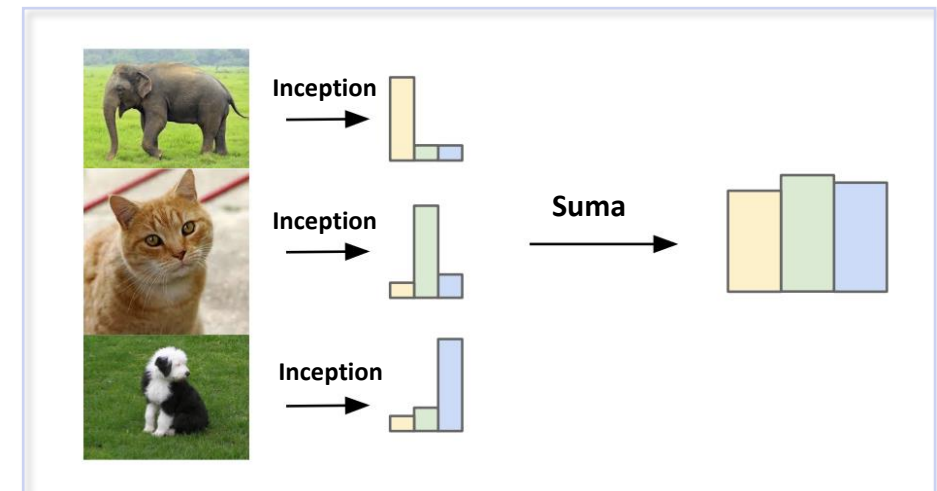
Cada imagen generada se pasa por una red Inception-v3. Devuelve una probabilidad de clase:

$$p(\text{clase}_j \mid \text{imagen}_i)$$

Divergencia Kullback–Leibler

A mayor diferencia, mayor el valor de score.

A más grande el score = hay diversidad e imágenes parecen reales



Probabilidad marginal (mide la diversidad)

$$p(\text{clase}) = \frac{1}{N} \sum_{i=1}^N p(\text{clase} \mid \text{imagen}_i)$$

- **Frechet Inception Distance (FID)**. Mide la distancia entre las distribuciones de las características extraídas de las imágenes reales y las imágenes generadas, usando una red preentrenada. Un valor de FID más bajo indica que las imágenes generadas están más cerca de las imágenes reales (para VAE y GAN).

$$FID = ||\mu_{real} - \mu_{gen}||^2 + Tr(\Sigma_{real} + \Sigma_{gen} - 2\sqrt{\Sigma_{real}\Sigma_{gen}})$$

- **SSIM (Structural Similarity Index)**. El SSIM es una métrica perceptual que mide la similitud entre dos imágenes considerando brillo, contraste y estructura. Esta métrica cuantifica qué tan bien el modelo preserva las características visuales esenciales durante el proceso de reconstrucción (para VAE).

$$SSIM(x, y) = \frac{(2\mu_{real}\mu_{gen} + C1)(2\sigma_{real,gen} + C2)}{(\mu_{real}^2 + \mu_{gen}^2 + C1)(\sigma_{real}^2 + \sigma_{gen}^2 + C2)} \quad \text{Se requieren pares (real, generada)}$$

- **Evaluación cualitativa.**
 - **Evaluación subjetiva o perceptual humana**. Se pide a evaluadores humanos juzgar directamente las imágenes generadas. Se muestran pares o grupos de imágenes (reales y generadas) y se pregunta, por ejemplo: ¿Cuál parece más real? (realismo), ¿Qué tan natural parece esta imagen? (coherencia semántica), ¿Cuál se prefiere? (preferencia estética), entre otras.
 - **Test de Turing**. Se muestran imágenes reales y generadas mezcladas. Los participantes deben decir cuáles creen que son reales. Se calcula el porcentaje de confusión.
 - **Mean Opinion Score (MOS)**. Método muy usado en visión, audio y video. Los evaluadores califican cada muestra en una escala subjetiva de calidad (por ejemplo, del 1 al 5).

¿Y si son textos?

- BLEU (Bilingual Evaluation Understudy). Se basa en la precisión de los n-gramas (secuencias contiguas de palabras) presentes en el texto generado en comparación con el texto de referencia.

Ejemplo:

Texto de referencia: “El gato está sobre la alfombra.” $\Rightarrow$ (El gato), (gato está), (está sobre), (sobre la), (la alfombra)

Texto generado: “Un gato se sienta en la alfombra.” $\Rightarrow$ (Un gato), (gato se), (se sienta), (sienta en), (en la), (la alfombra)

Coincidencias: uni-gramas: “gato”, “la”, “alfombra” (3 coincidencias)

bi-gramas: (la, alfombra) (1 coincidencias)

¿Cómo se calcula?

- Se cuentan los n-gramas (en general, de $n = 1..4$) que aparecen tanto en el texto generado como en el de referencia.
- Se determinan cuántos n-gramas coinciden y se divide entre el total de n-gramas.
- Estas precisiones se combinan de la siguiente manera:

$$BLEU = BP \times \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

$BP = e^{(1-r/c)}$ si $c < r$ (penalización por longitud)
 p_n = precisión de los n-gramas
 w_n = peso de cada n-grama
 r = longitud del texto de referencia
 c = longitud del texto generado

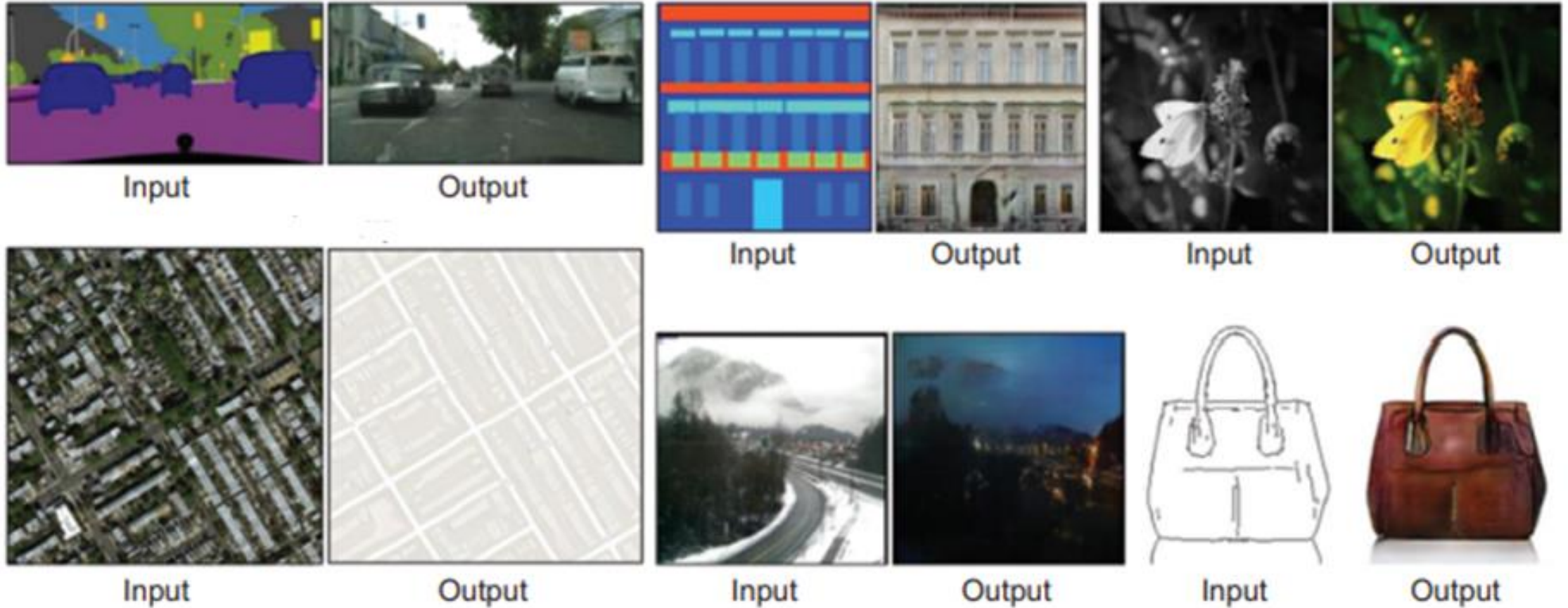
Interpretación: medida de similitud entre los dos textos. Varía de 0 a 1
BLEU $\approx$ 1: muy parecidos.
BLEU $\approx$ 0: muy diferentes.

- Otras: METEOR, ROUGE, CIDEr. Se pueden combinar, incluyendo evaluaciones humanas.

❑ Algunas aplicaciones de las GAN

“Traducción” entre dominios (*cross-domain translation*)

Traducción imagen a imagen (*imagen-to-image translation*)

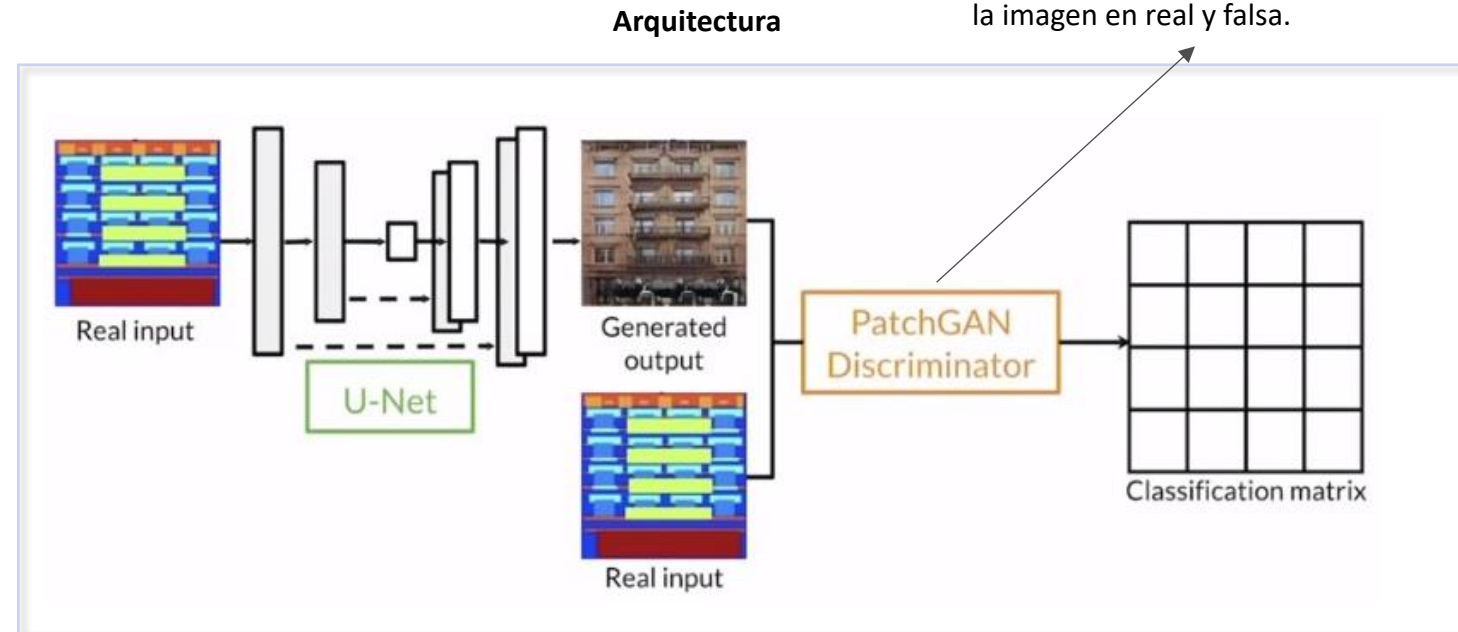
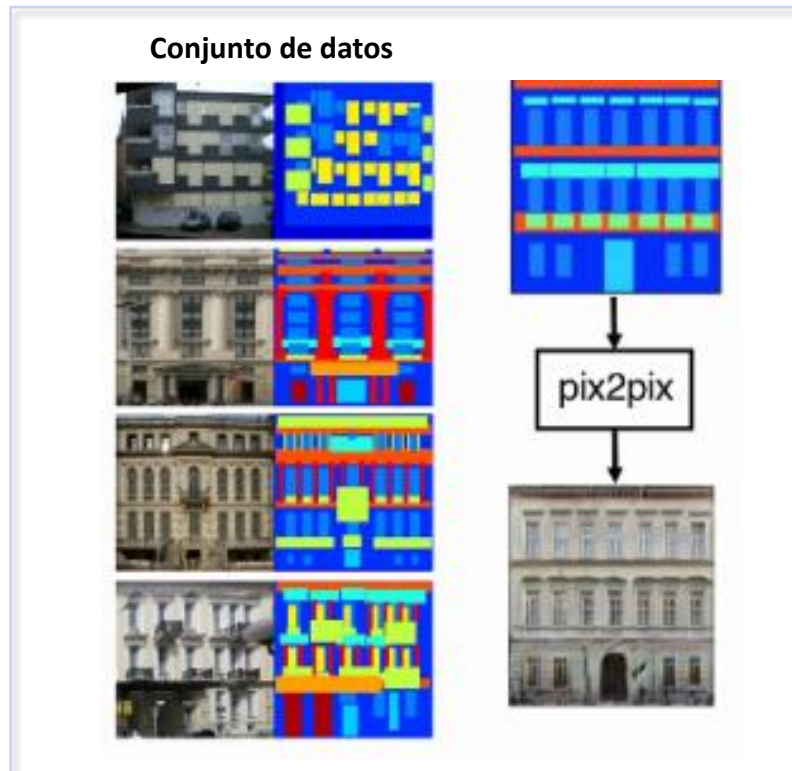


Se copia el estilo de un conjunto de imágenes de referencia en otra imagen.

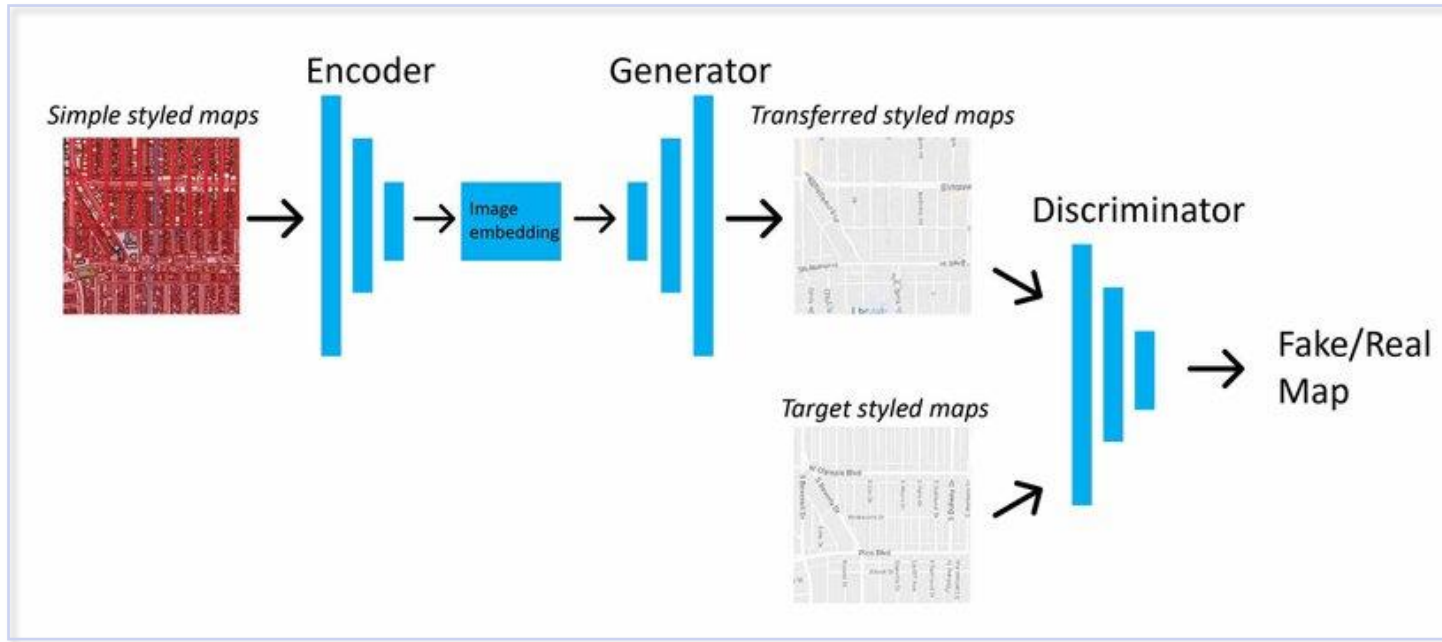
- Ejemplos: {
- **Pix2pix:** *dominio A* $\rightarrow$ *dominio B*
 - **CycleGAN:** *dominio A* $\rightleftarrows$ *dominio B*

Pix2pix:

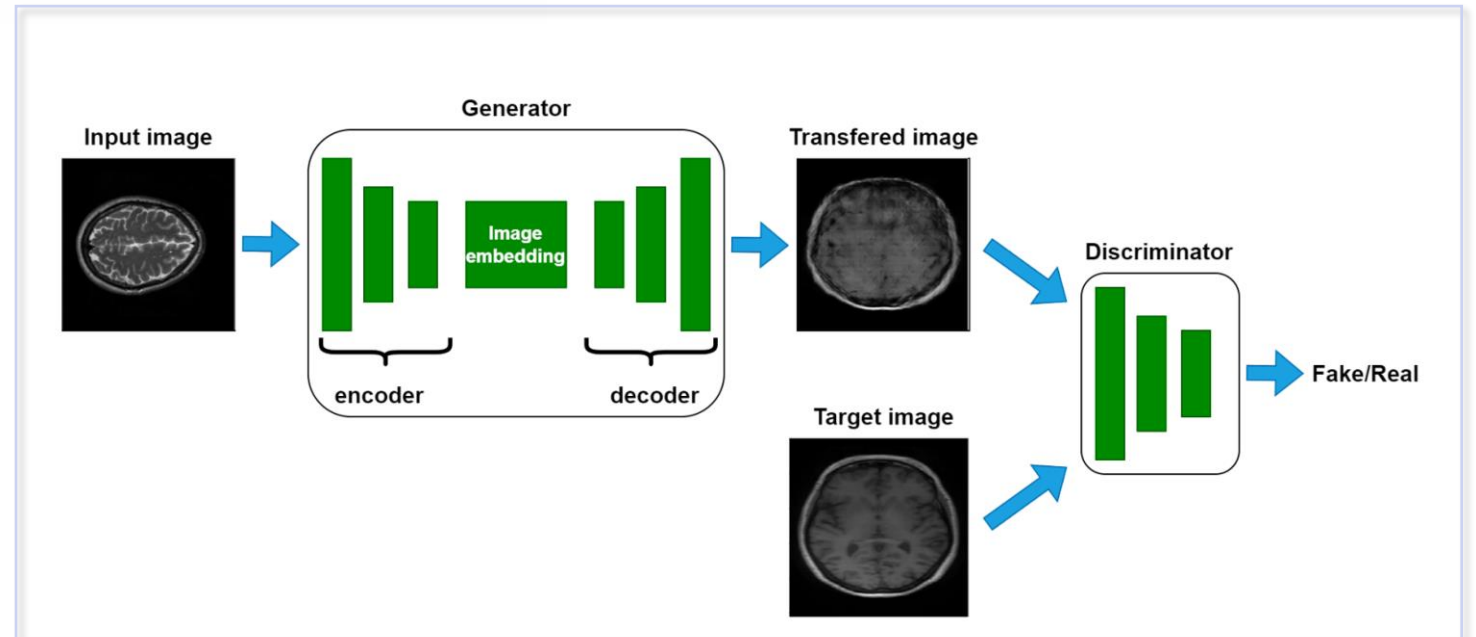
- Se basa en una arquitectura GAN condicional.
- Para el entrenamiento se requiere pares de datos:



Ejemplos de uso:



<https://developers.arcgis.com/python/guide/how-pix2pix-works/>



Generating Synthetic Images for Healthcare
with Novel Deep Pix2Pix GAN
(<https://www.mdpi.com/2079-9292/11/21/3470>)

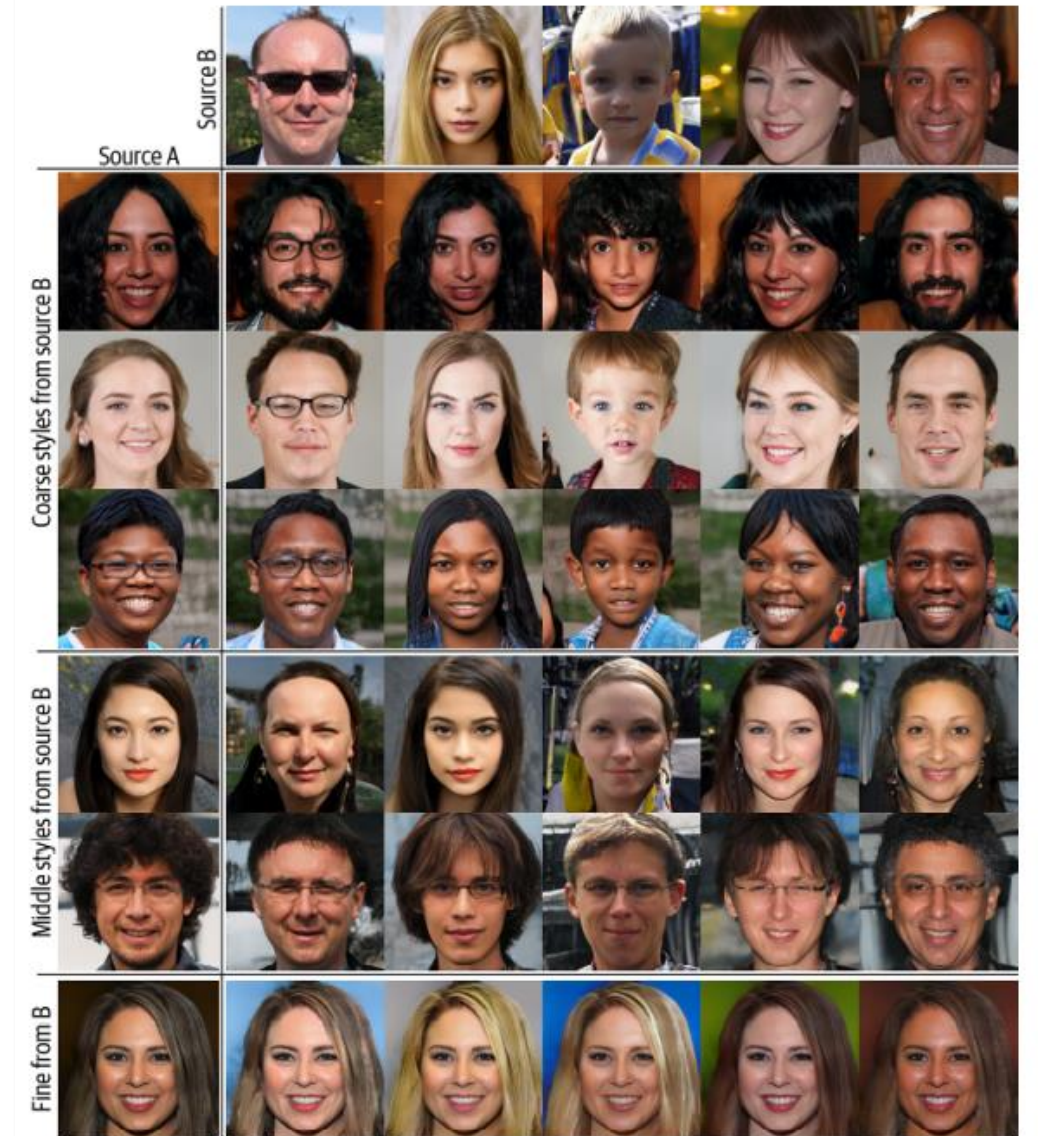
Transferencia de estilo

Copiar el estilo de una imagen en otra imagen.

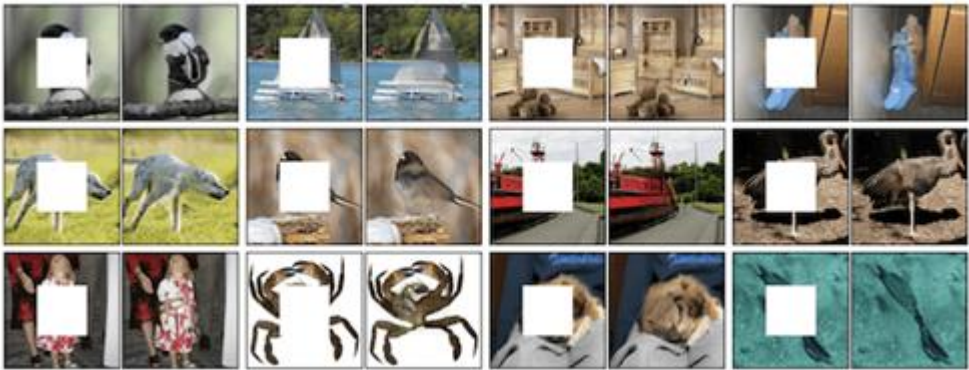


■ Ejemplo: StyleGAN

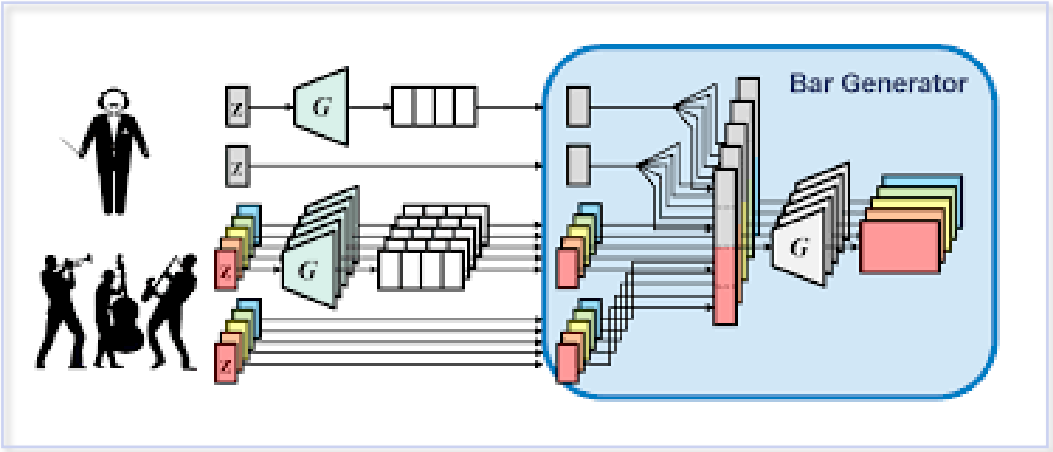
- Función de costo compuesta:
 1. Conten loss. ¿La imagen combinada tenga el mismo contenido que la imagen base?
 2. Style loss. ¿La imagen combinada tiene el mismo estilo general que la imagen base?
 3. Total variance loss. ¿La imagen combinada tiene un aspecto suave y no pixelado?



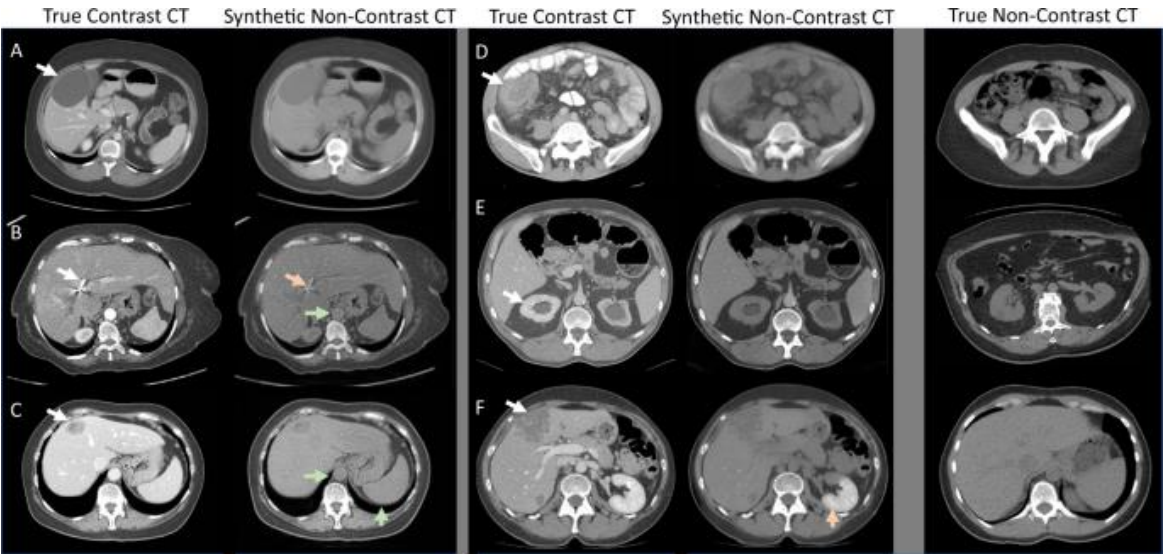
18 Impressive Applications of Generative Adversarial Networks (GANs)
(<https://machinelearningmastery.com/impressive-applications-of-generative-adversarial-networks/>)



Data augmentation using generative adversarial networks (CycleGAN) to improve generalizability in CT segmentation tasks
(<https://www.nature.com/articles/s41598-019-52737-x>)



MuseGAN: Multi-track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment
(<https://arxiv.org/abs/1709.06298>)



- Procesamiento de Lenguaje Natural.
- Modelos secuenciales.
- Redes recurrentes, LSTM y GRU.